DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING LAB MANUAL

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Name: TARANYA.M

Register No: 241001285

Year / Branch / Section: II / B.Tech IT

Semester: IV

Academic Year: 2025-2026

INDEX

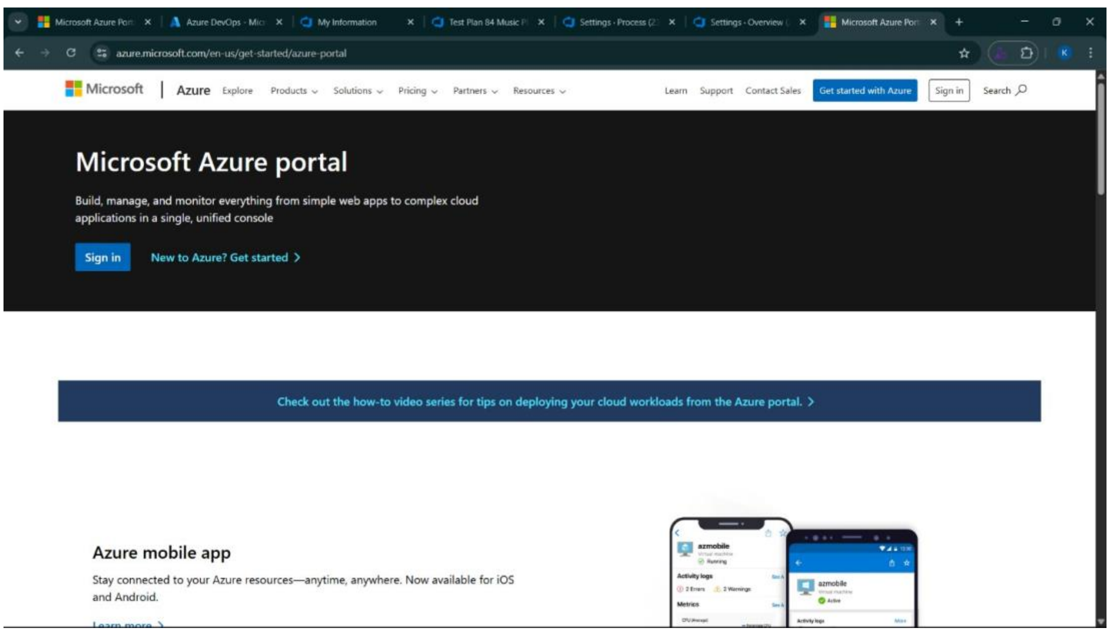
EXP NO: 1	AZURE DEVOPS ENVIRONMENT SETUP

Aim:

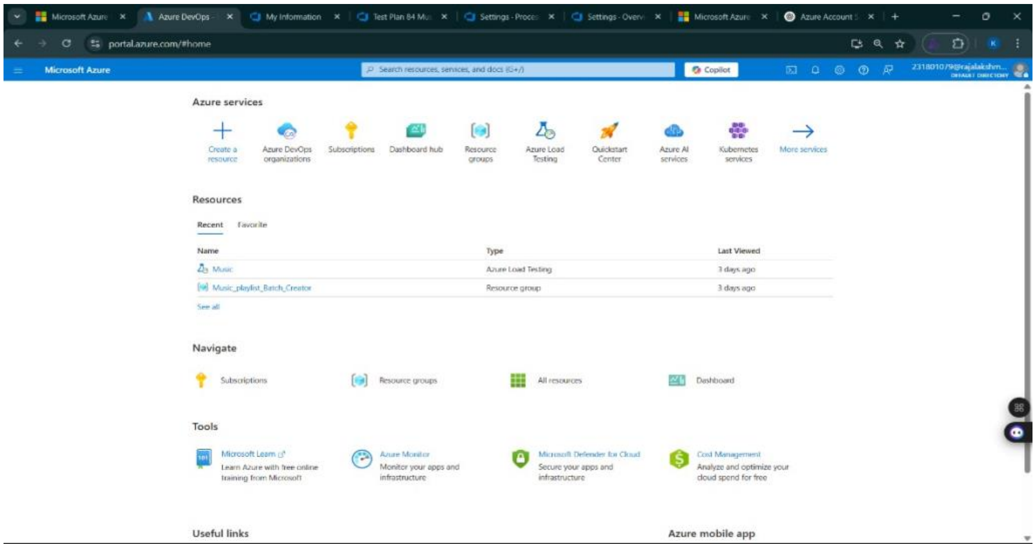
To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION

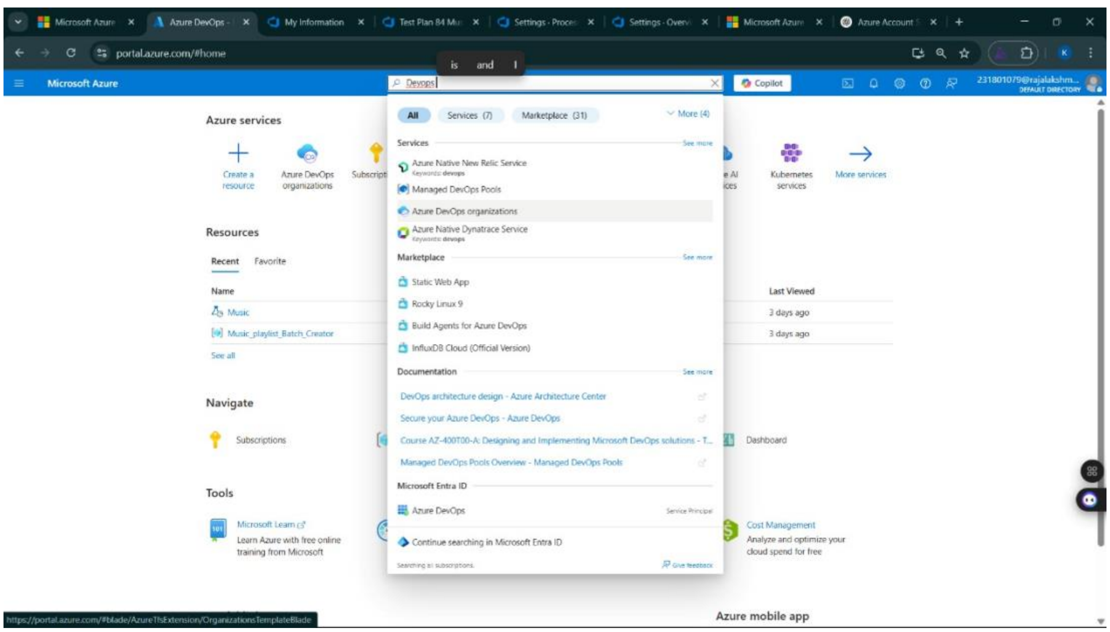
1.Open your web browser and go to the Azure website: https://azure.microsoft.com/en-us/get-started/azure-portal.
Sign in using your Microsoft account credentials.
If you don't have a Microsoft account, you can create one here: https://signup.live.com/?lic=1



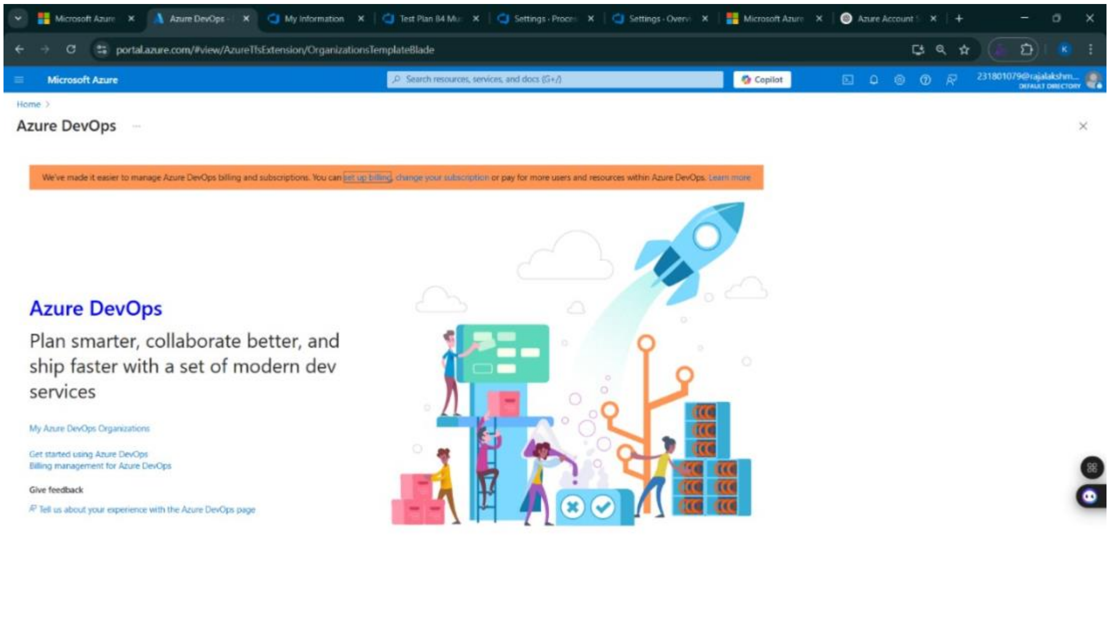
2.Azure home page



3.Open DevOps environment in the Azure platform by typing ***Azure DevOps Organizations*** in the search bar.



4.Click on the ***My Azure DevOps Organization*** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



Result:

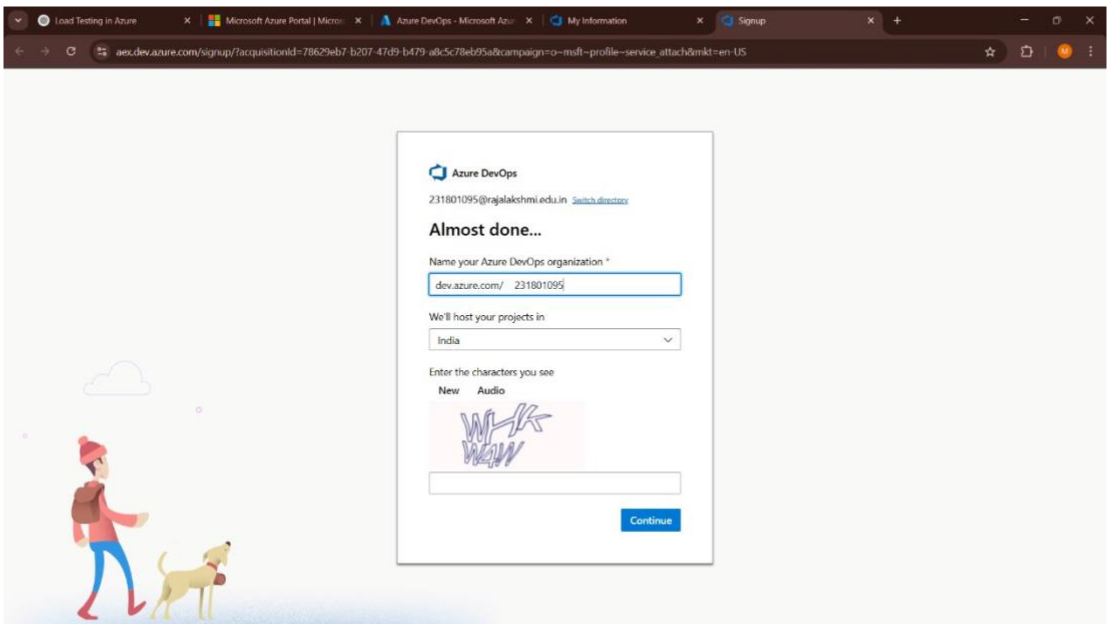
Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXP NO: 2	AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

1.Create An Azure Account



2.Create the First Project in Your Organization

a. After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.

b. On the organization's **Home page**, click on the **New Project** button.

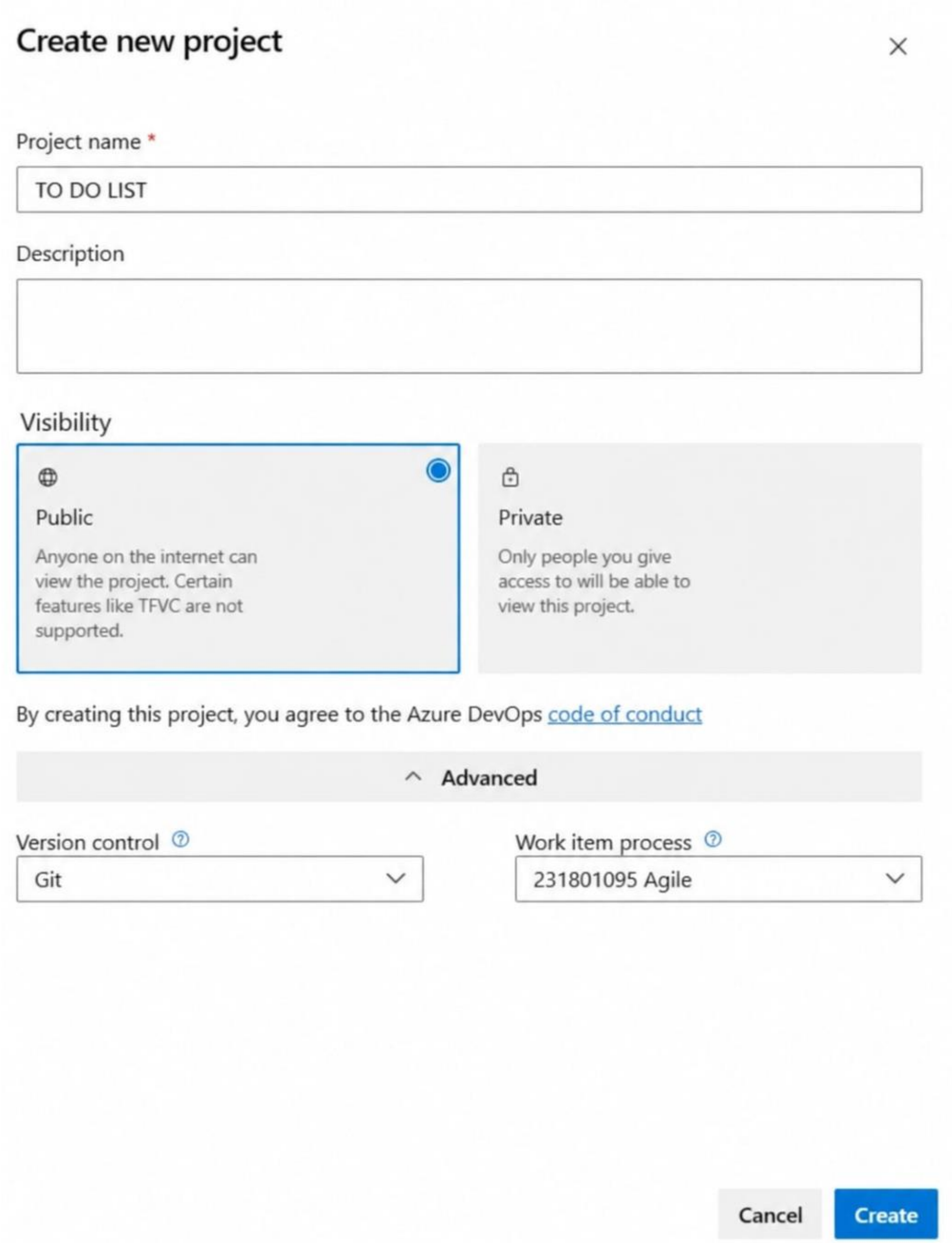
c. Enter the project name, description, and visibility options:

***Name**:* Choose a name for the project (e.g., To-Do List App).

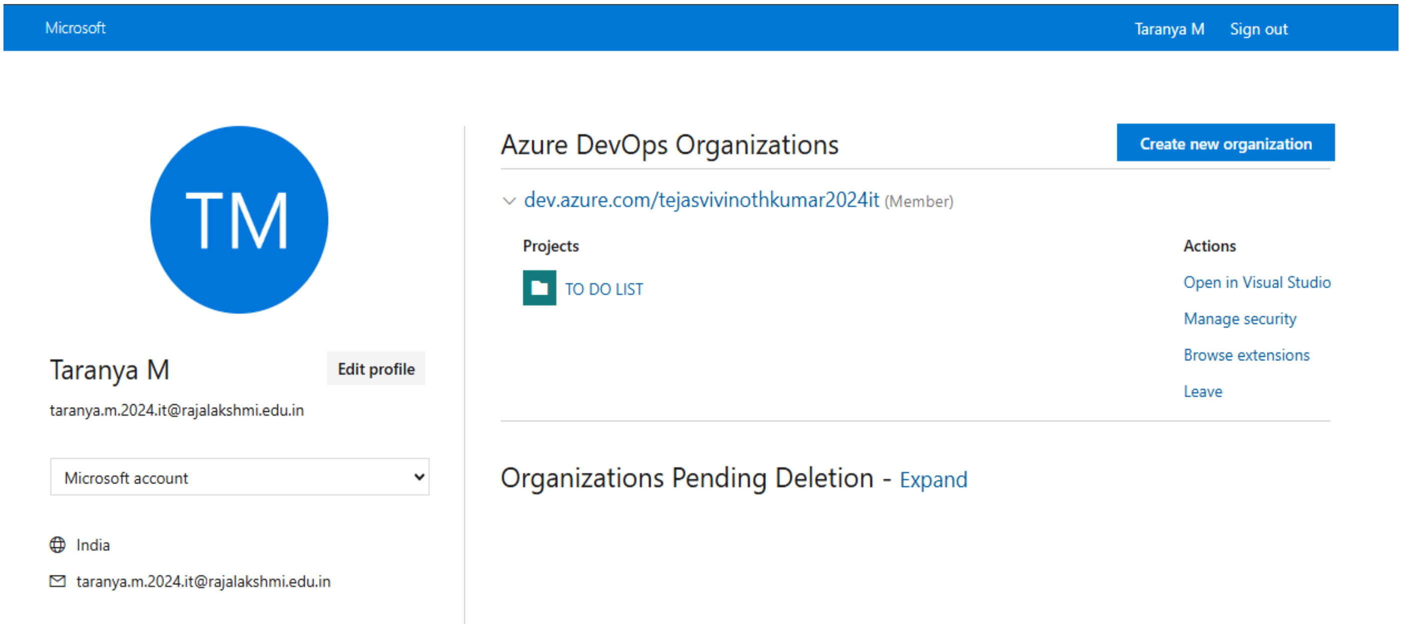
***Description**:* Optionally, add a description to provide more context about the project.

***Visibility**:* Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).

d. Once you've filled out the details, click **Create** to set up your first project.



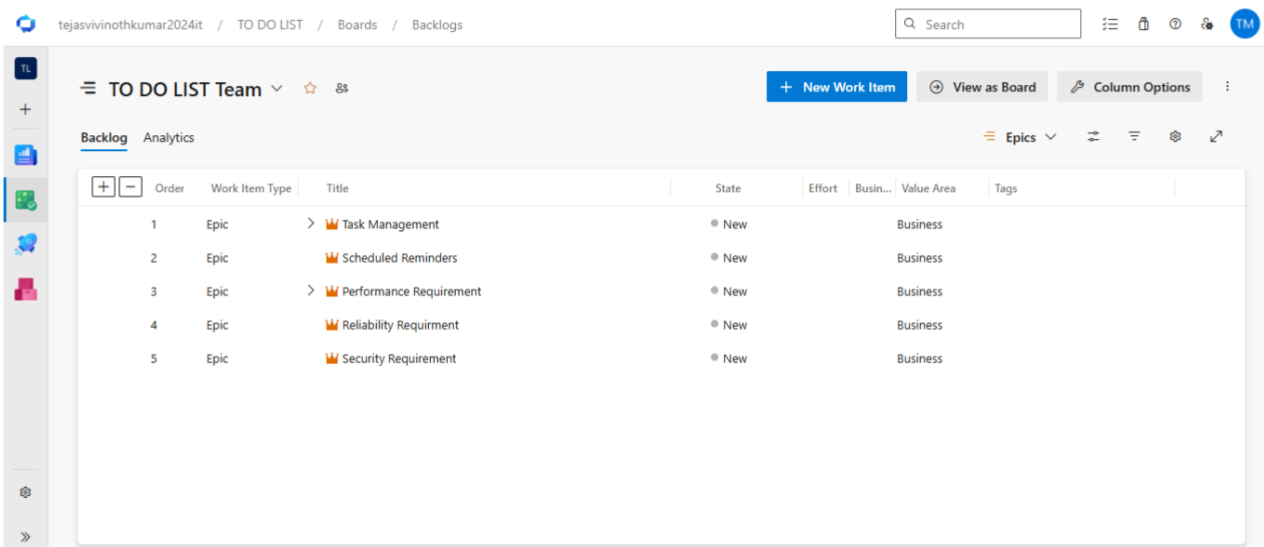
3.Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



4.To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.



Result:

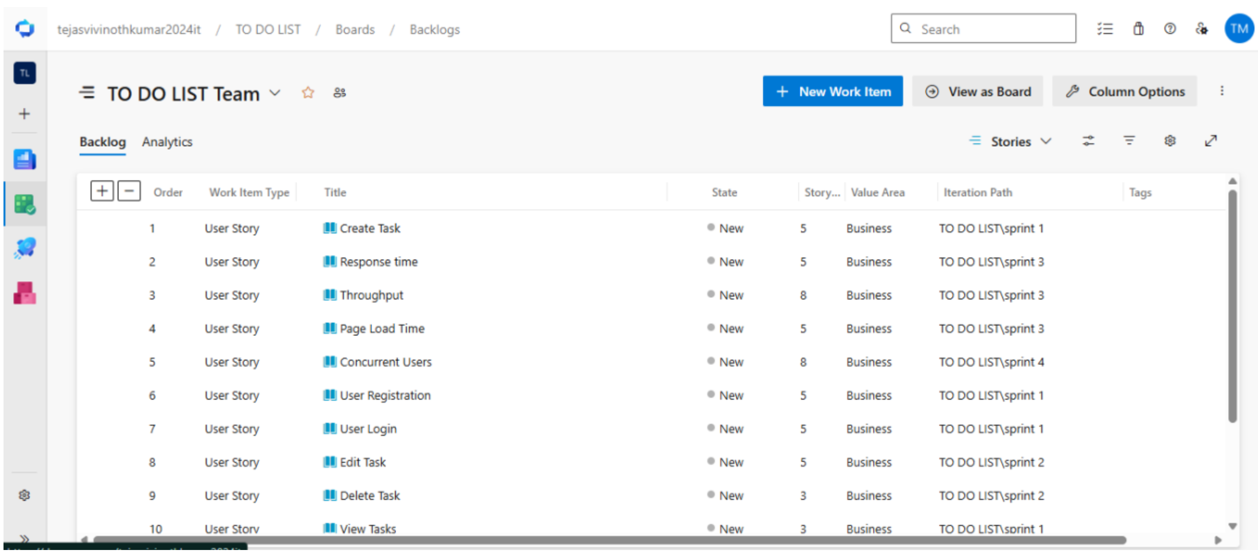
Successfully created an Azure DevOps project with user story management and agile workflow setup.

	SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

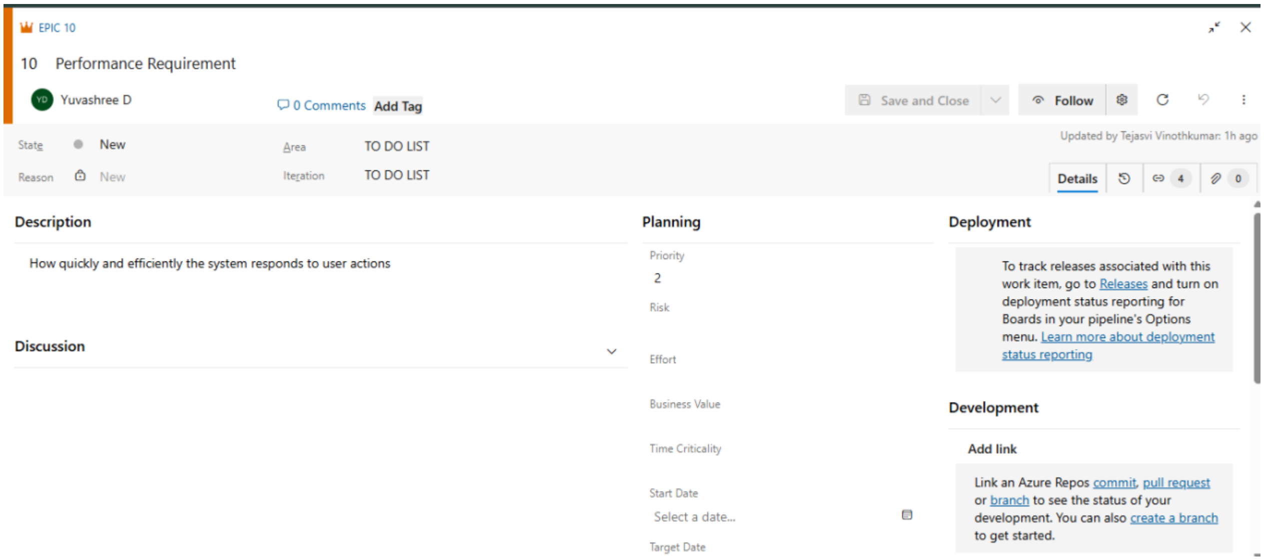
Aim:

To learn about how to create epics, user story, features, backlogs for your assigned project.

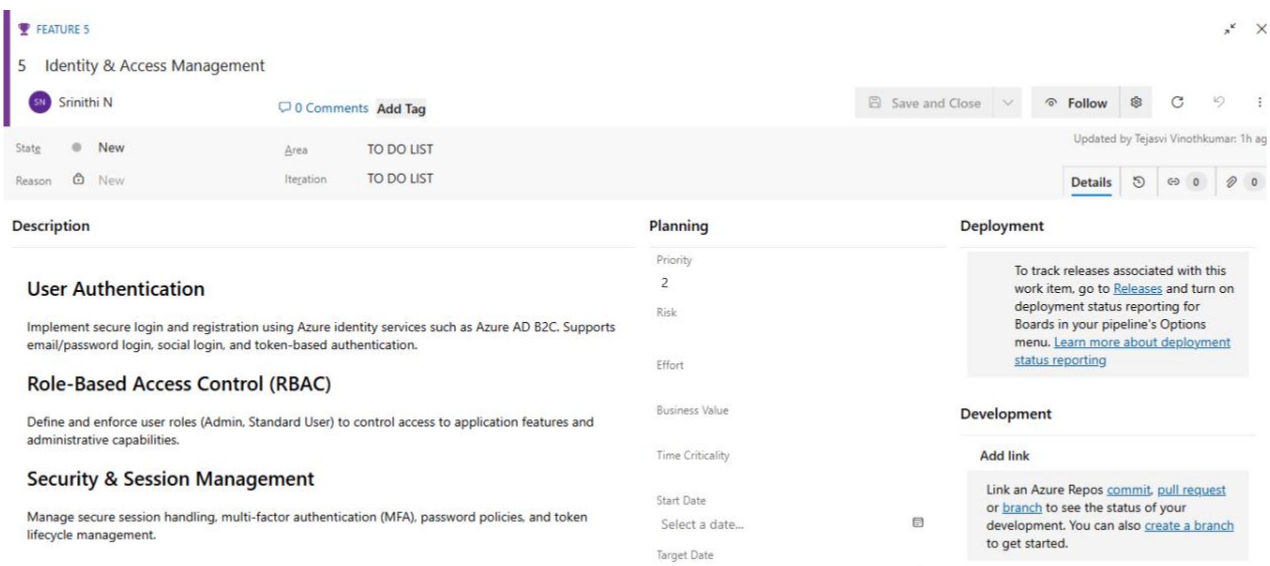
Create Epic, Features, User Stories, Task



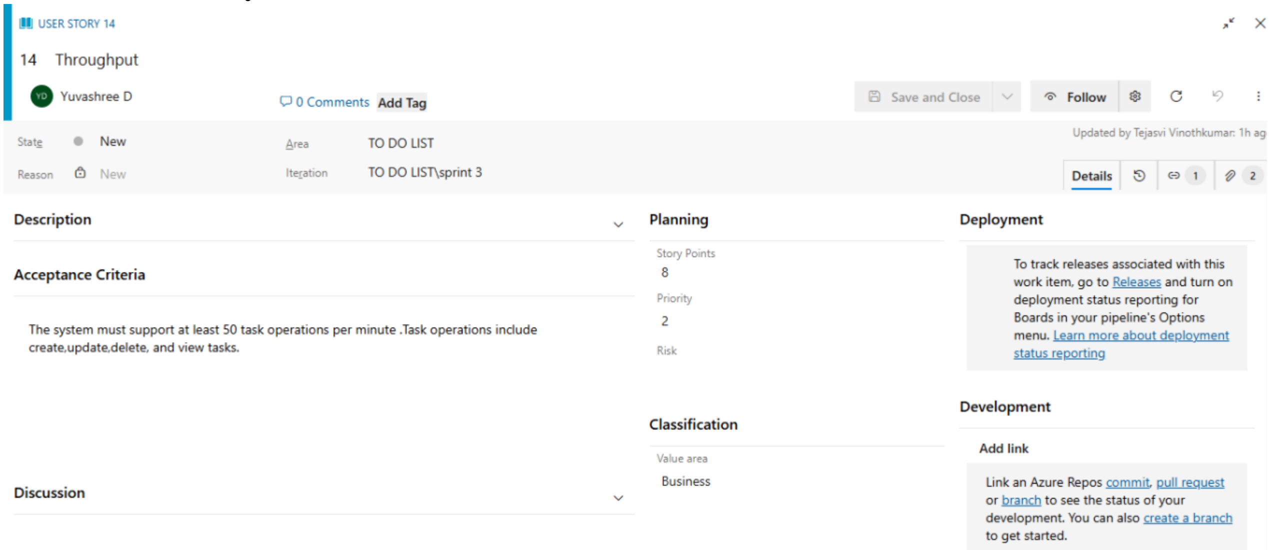
1.Fill in Epics



2.Fill in Features



3.Fill in User Story Details



Result:

Thus, the creation of epics, features, user story and task has been created successfully.

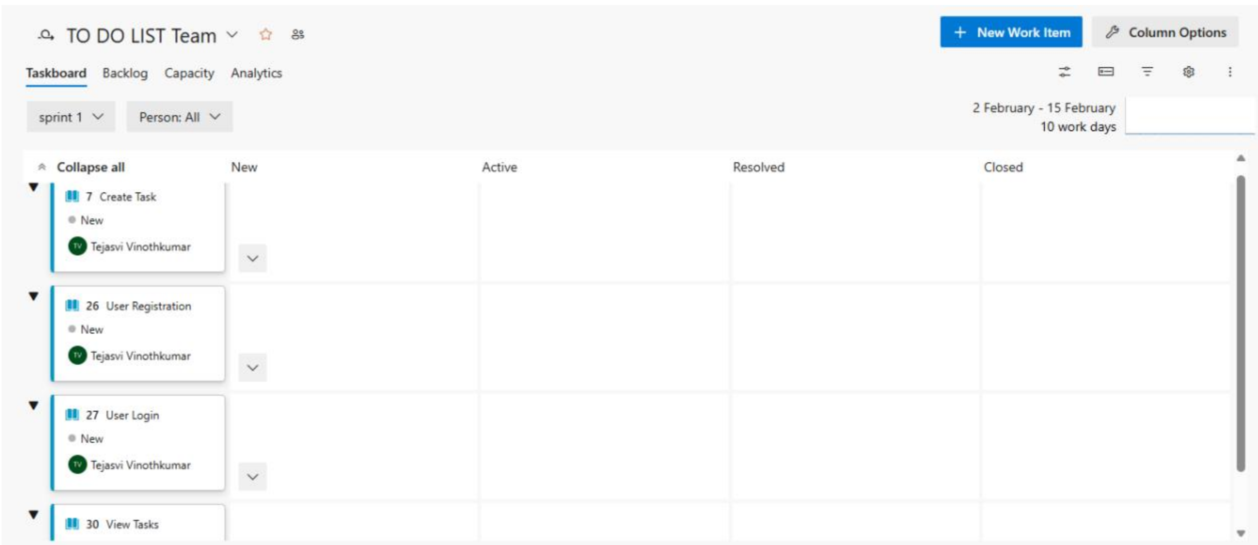
SPRINT PLANNING

Aim:

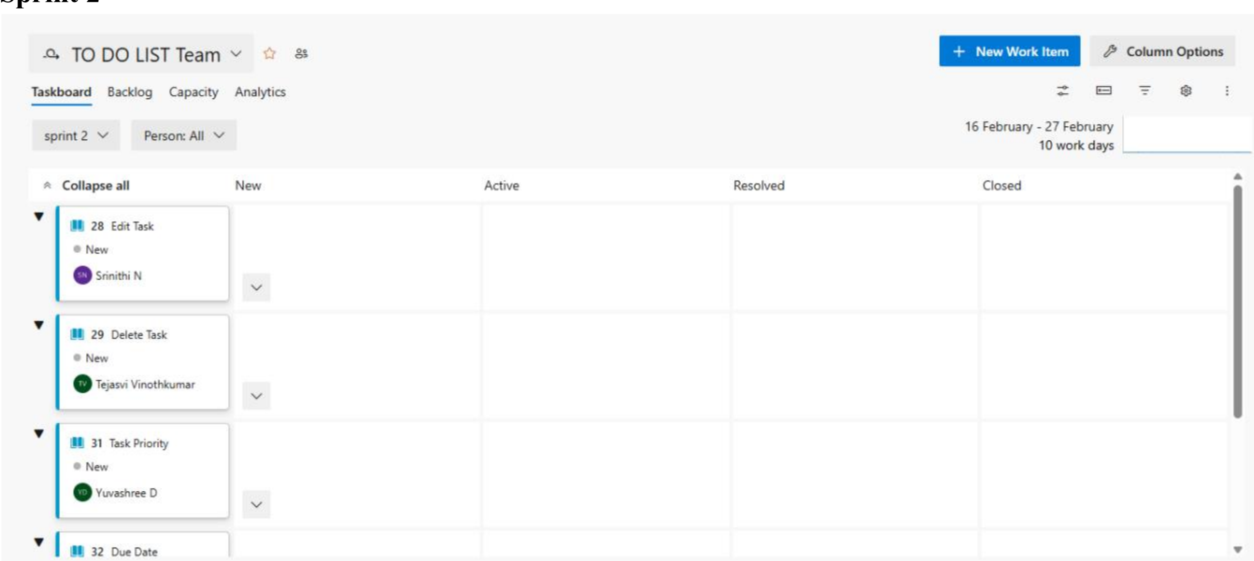
To assign user story to specific sprint for the Advanced Reliable To-Do List Application.

Sprint Planning

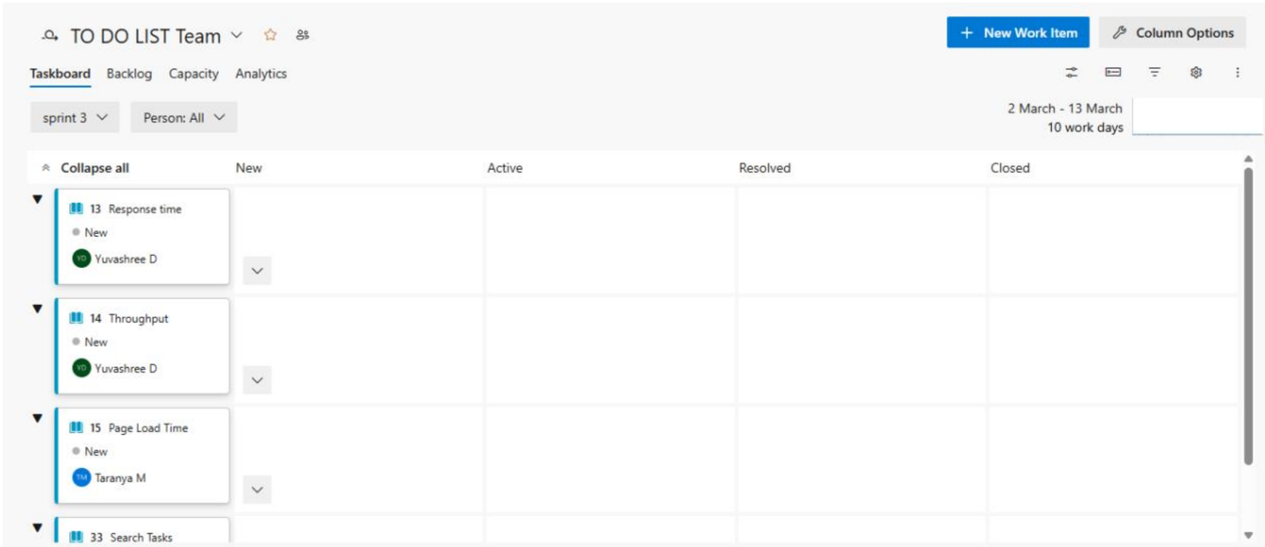
Sprint 1



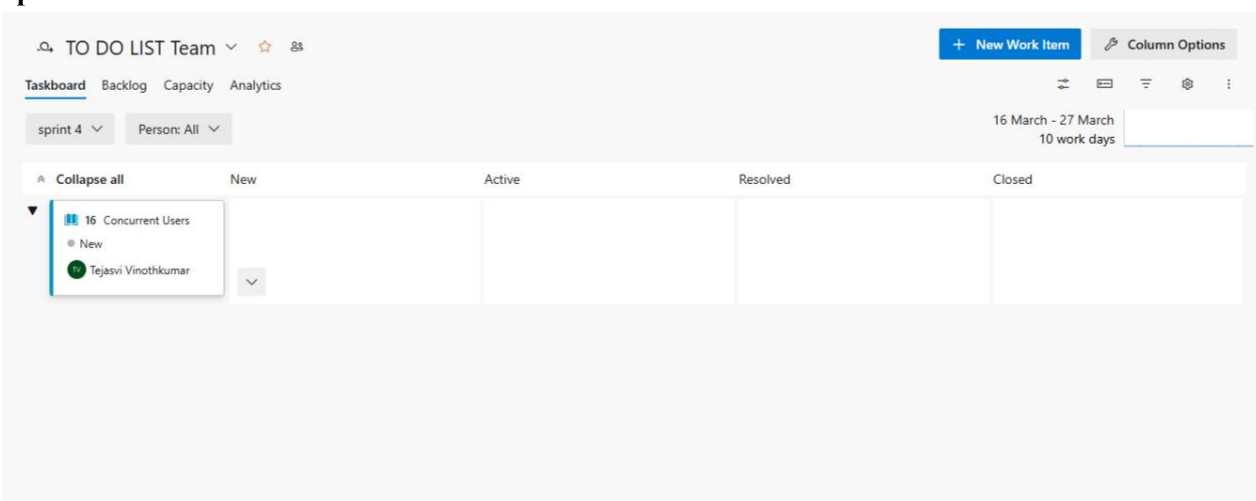
Sprint 2



Sprint 3



Sprint 4



Result:

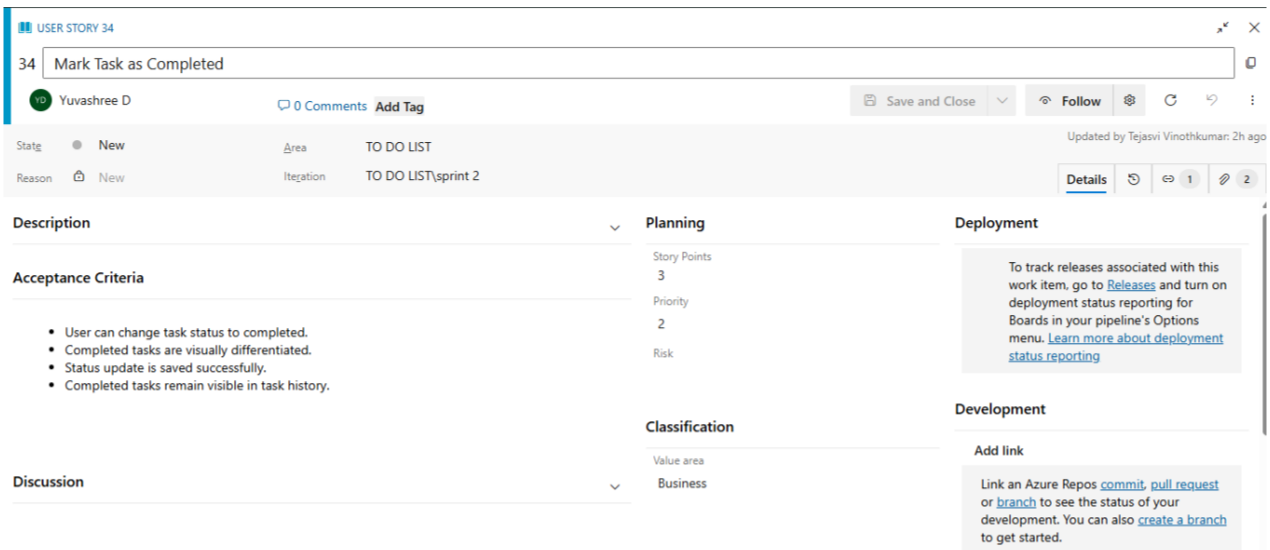
The Sprints are created for the To-Do List Application.

POKER ESTIMATION

Aim:

Create Poker Estimation for the user stories - Music Playlist Batch Creator Project.

Poker Estimation



Result:

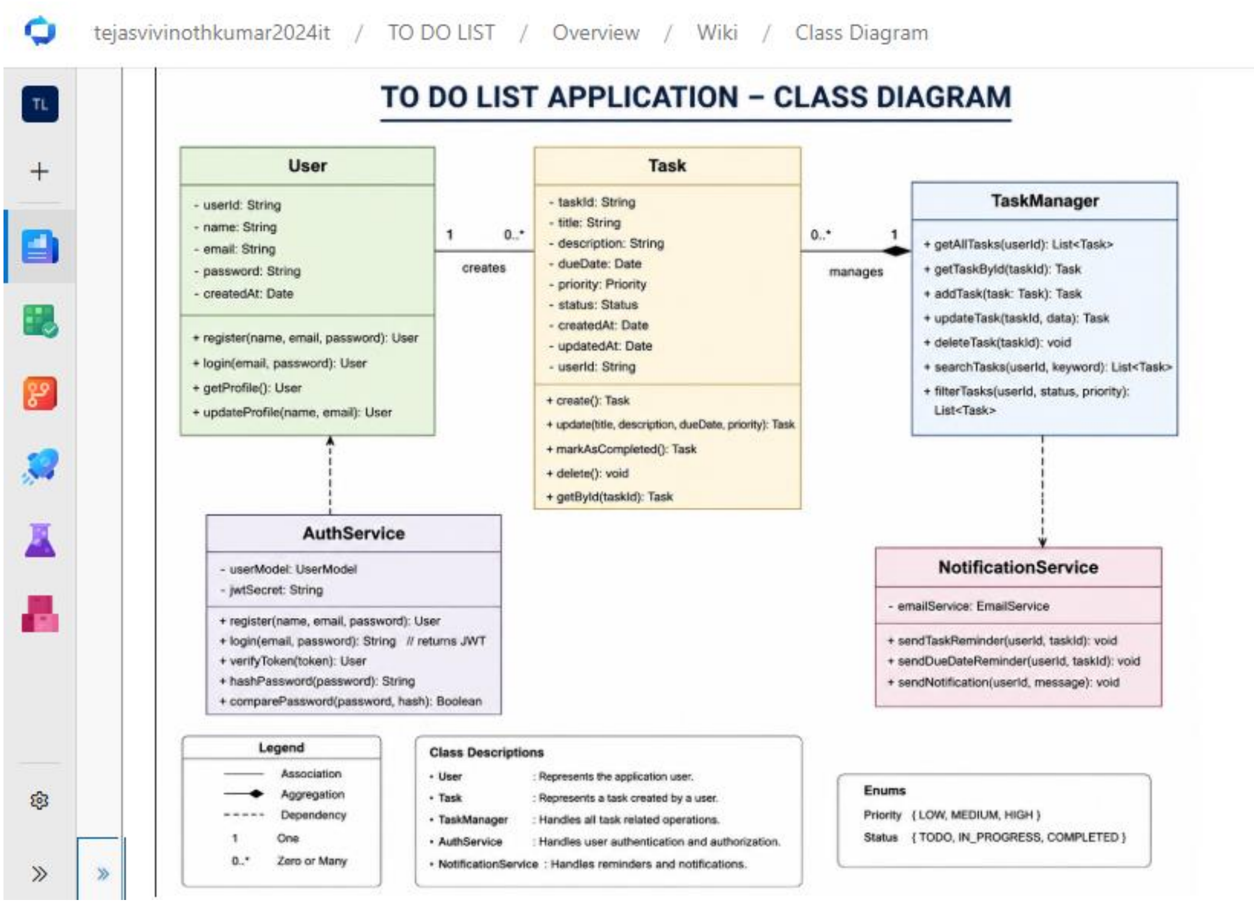
The Estimation/Story Points is created for the project using Poker Estimation.

DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE

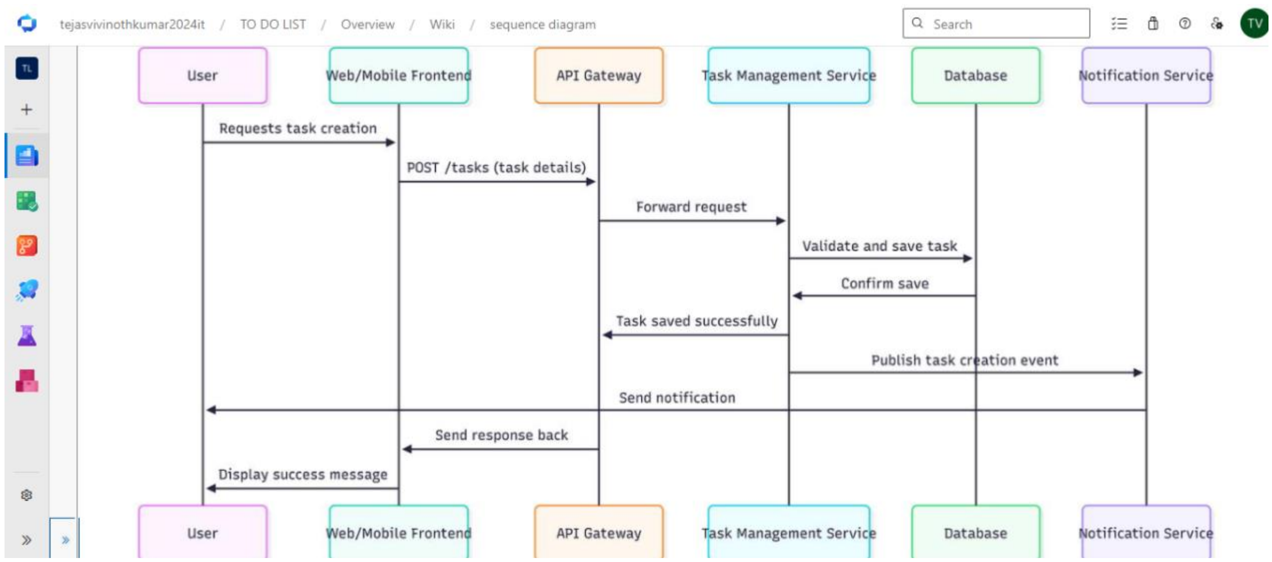
Aim:

To Design a Class Diagram and Sequence Diagram for the given Project.

6A. Class Diagram



6B. Sequence Diagram



Result:

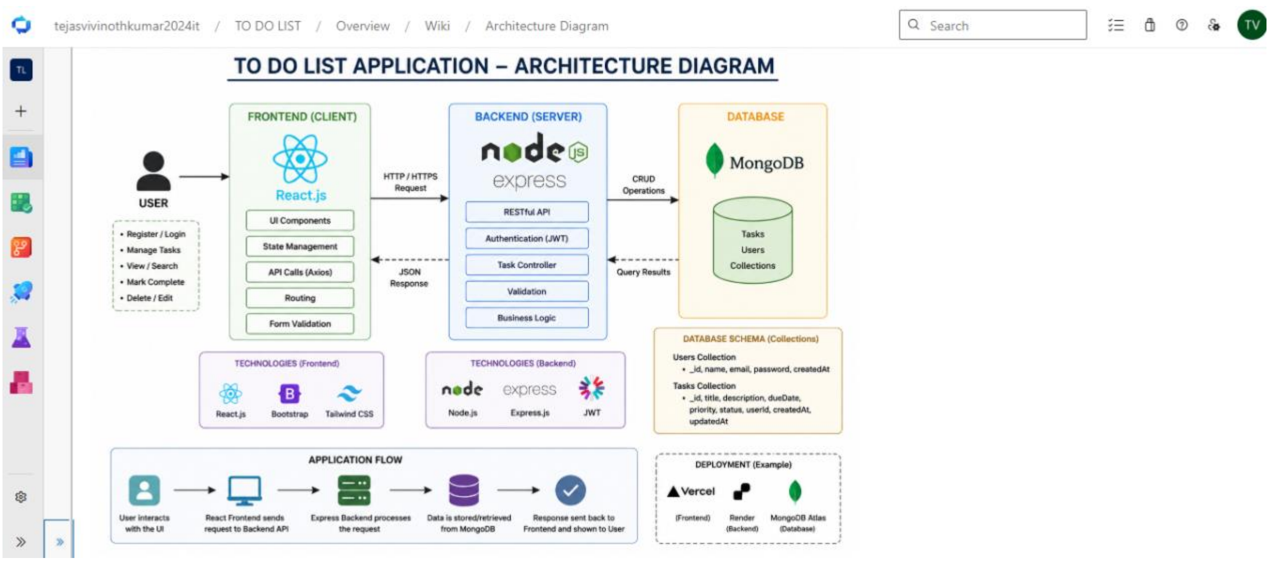
The Class Diagram and Sequence Diagram is designed Successfully for the To-Do List Application.

DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE

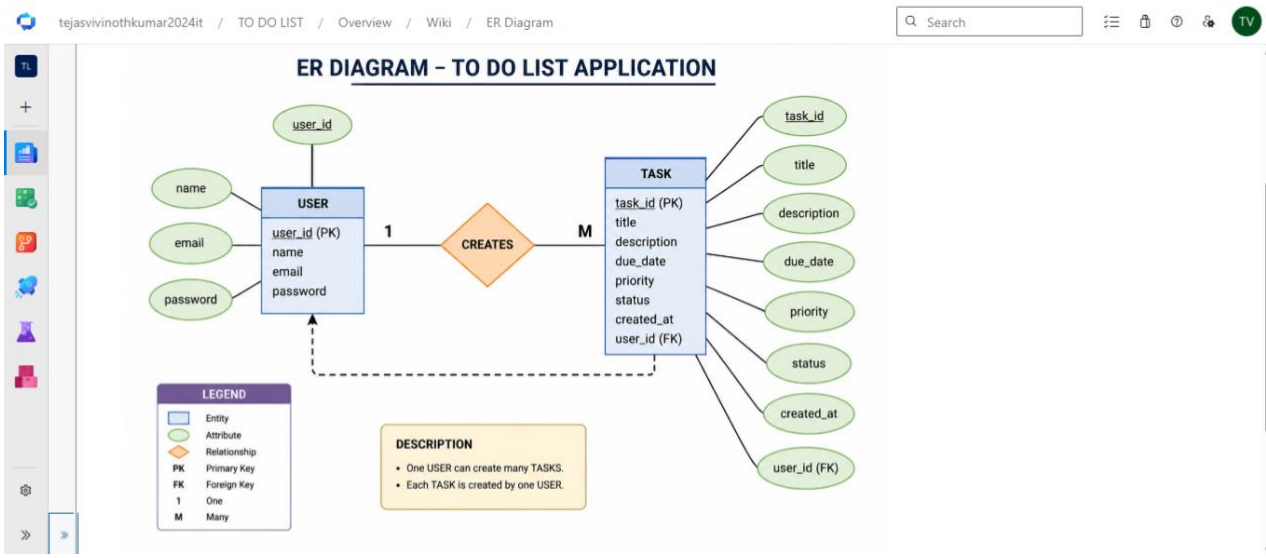
Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7A. Architectural Diagram



7B.ER Diagram



Result:

The Architecture Diagram and ER Diagram is designed Successfully for the To-Do List Application.

	TESTING – TEST PLANS AND TEST CASES

Aim:

Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case

Test Case Design Procedure

1) Understand Core Features of the Application

- User Registration & Login
- Create, Update, Delete Tasks
- Task Prioritization & Filtering
- Mark Task as Completed
- Notifications / Reminders
- Data Sync with Cloud (Azure)

2) Define User Interactions

Each test case simulates real user behavior

Eg: logging in, adding tasks, editing tasks, marking complete.

3) Design Happy Path Test Cases

Validates that all features work under normal conditions

Eg: User logs in → creates task → marks it complete successfully

4) Design Error Path Test Cases

Tests invalid or unexpected scenarios

Eg: login fails with wrong password, task creation fails with empty input

5) Break Down Steps and Expected Results

Each test case includes:

Step-by-step actions

Expected output

6) Use Clear Naming and IDs

Eg:

TC01 – Successful Login

TC02 – Login Failure

7) Separate Test Suites

Group test cases based on functionality:

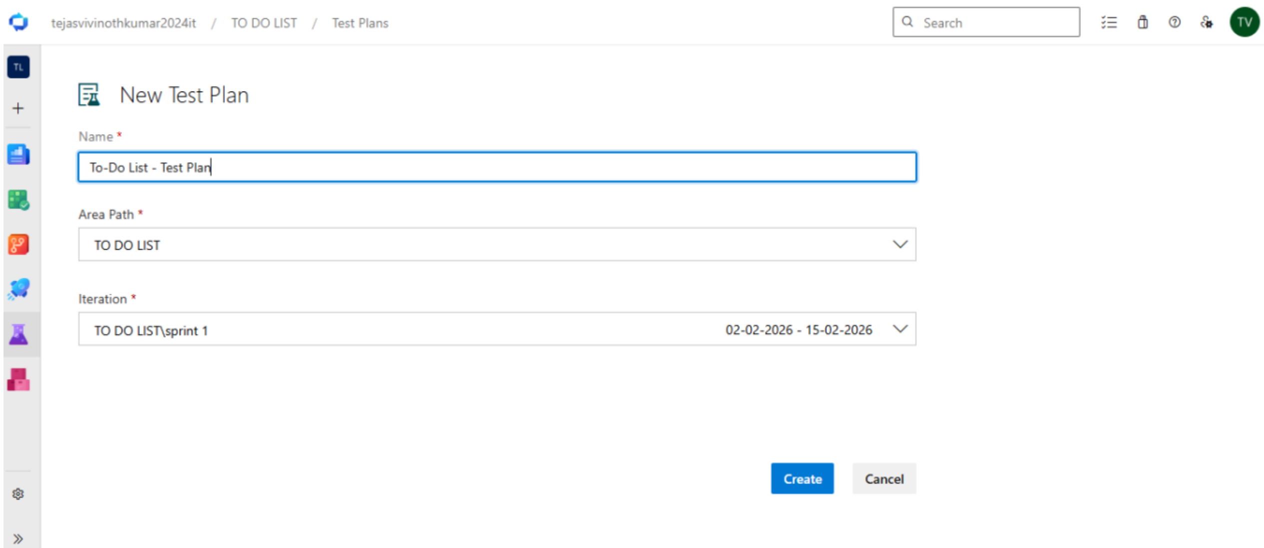
(Login, Task Management, Notifications)

8) Prioritize and Review

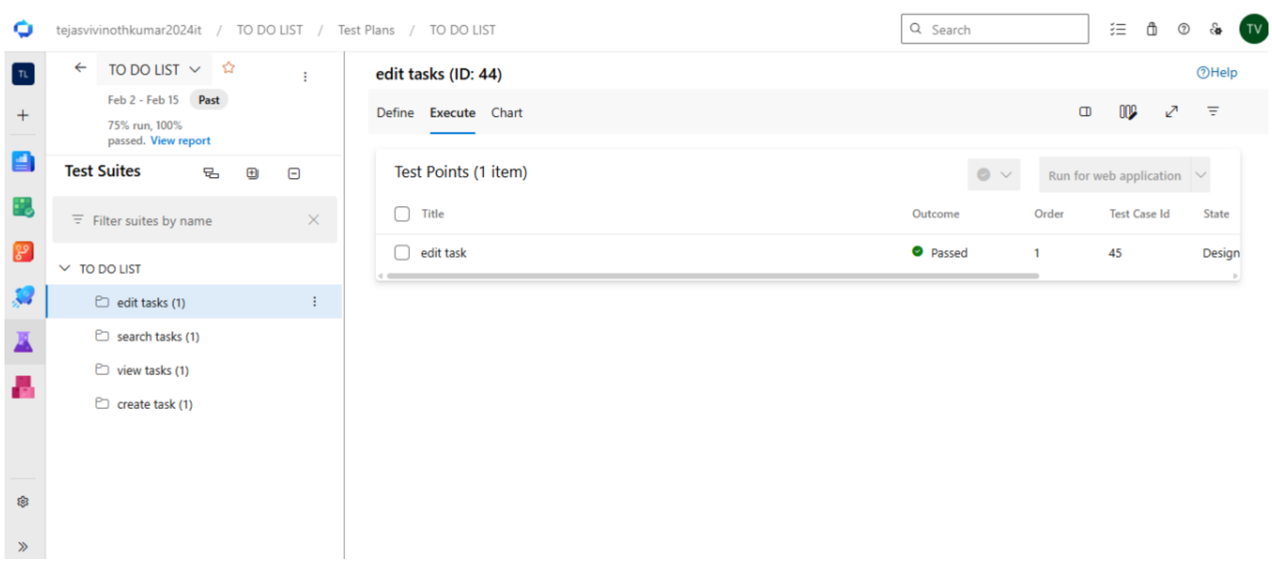
Critical features → High priority

Ensure completeness and accuracy

1.New test plan



2.Test suite



3.Test case

Give two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

To-Do List Application – Test Plans

USER STORIES

- As a user, I want to register and log in so that I can manage my tasks
- As a user, I want to create and organize tasks
- As a user, I want to update or delete tasks
- As a user, I want to mark tasks as completed
- As a user, I want to receive reminders/notifications

Test Suites

Test Suite: TS01 – User Login (ID: 101)

1) TC01 – Successful Sign Up

Action:

- Go to Sign-Up page
- Enter valid details
- Click "Sign Up"

Expected Results:

- Account created successfully
- Redirected to dashboard
- Type: Happy Path

2) TC02 – Secure Login

Action:

- Go to Login page
- Enter valid credentials
- Click "Login"
- Expected Results:
- Login successful
- Dashboard displayed
- Type: Happy Path

3) TC03 – Sign Up with Existing Email

Action:

- Enter already registered email
- Click "Sign Up"
- Expected Results:
- Error message: "Email already exists"

- Type: Error Path

4) TC04 – Login with Wrong Password

Action:

- Enter incorrect password
- Click "Login"
- Expected Results:
- Error message displayed
- Type: Error Path

Test Suite: TS02 – Task Management (ID: 102)

1) TC05 – View Task List

Action:

- Login
- Open task dashboard
- Expected Results:
- List of tasks displayed
- Type: Happy Path

2) TC06 – Task Load Failure (No Internet)

Action:

- Turn off internet
- Open task list
- Expected Results:
- Error message displayed
- Type: Error Path

Test Suite: TS03 – Task Operations (ID: 103)

1) TC07 – Add Task

Action:

- Enter task details
- Click "Add Task"
- Expected Results:

- Task added successfully
- Type: Happy Path

2) TC08 – Add Task with Missing Data

Action:

- Leave fields empty
- Submit
- Expected Results:
- Validation error shown
- Type: Error Path

Test Suite: TS04 – Task Completion & Review (ID: 104)

1) TC09 – Mark Task as Completed

Action:

- Select a task
- Click "Mark as Done"
- Expected Results:
- Task marked as completed
- Type: Happy Path

2) TC10 – Mark Non-existing Task

Action:

- Try marking invalid/non-existing task
- Expected Results:
- Error message shown
- Type: Error Path

Test Suite: TS05 – Notifications (ID: 105)

1) TC11 – Receive Notification

Action:

- Set reminder for task
- Expected Results:
- Notification received via app/email

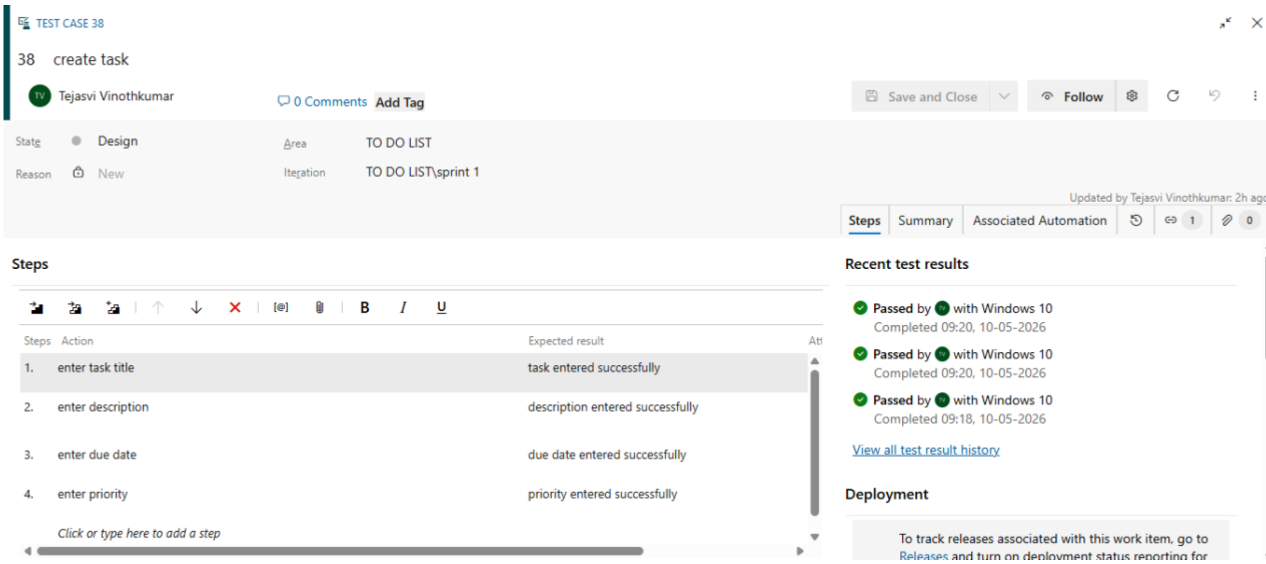
- Type: Happy Path

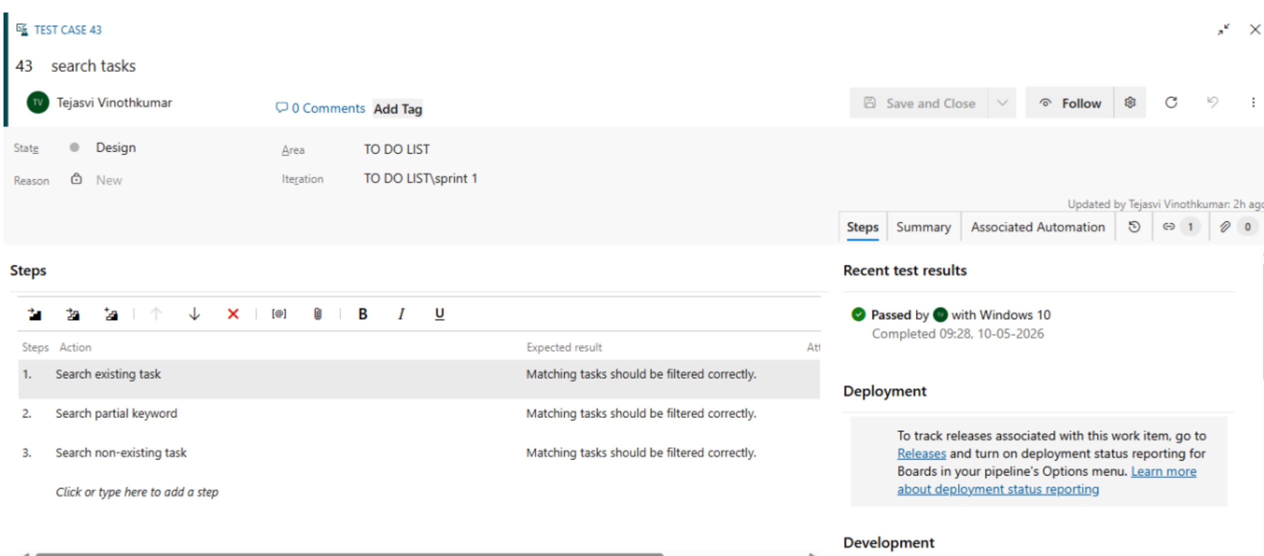
2) TC12 – Notification Failure

Action:

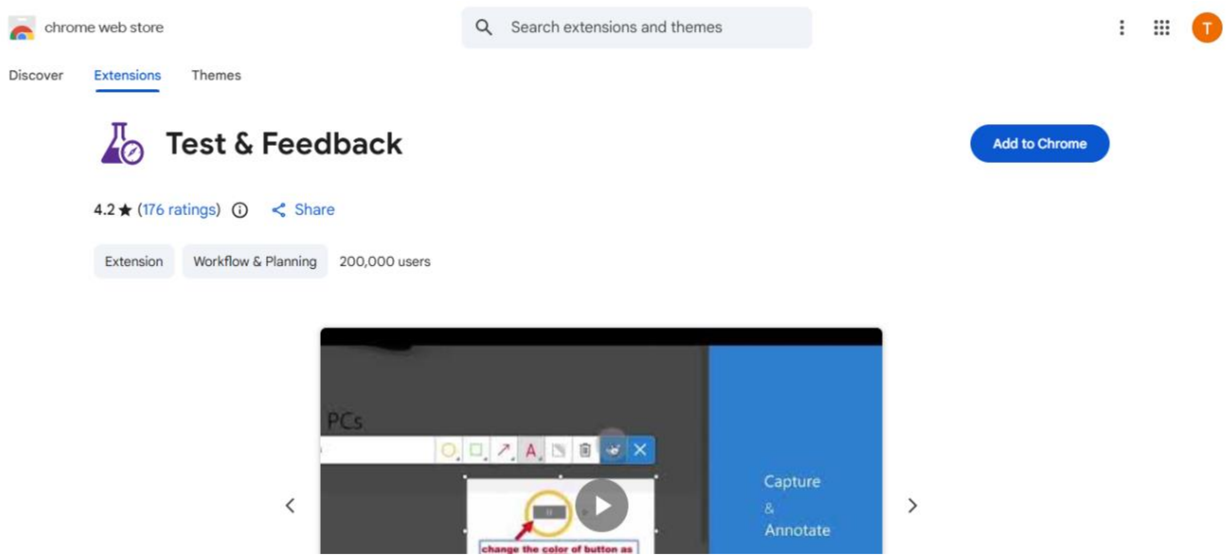
- Trigger notification with network issue
- Expected Results:
- Notification not sent / error logged
- Type: Error Path

Test Cases

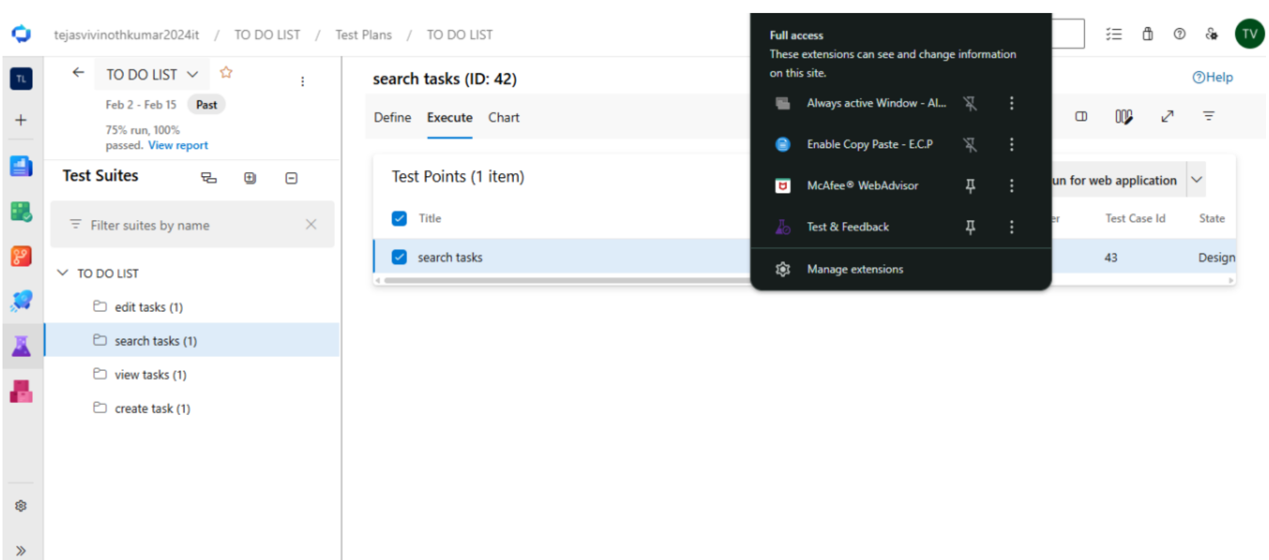




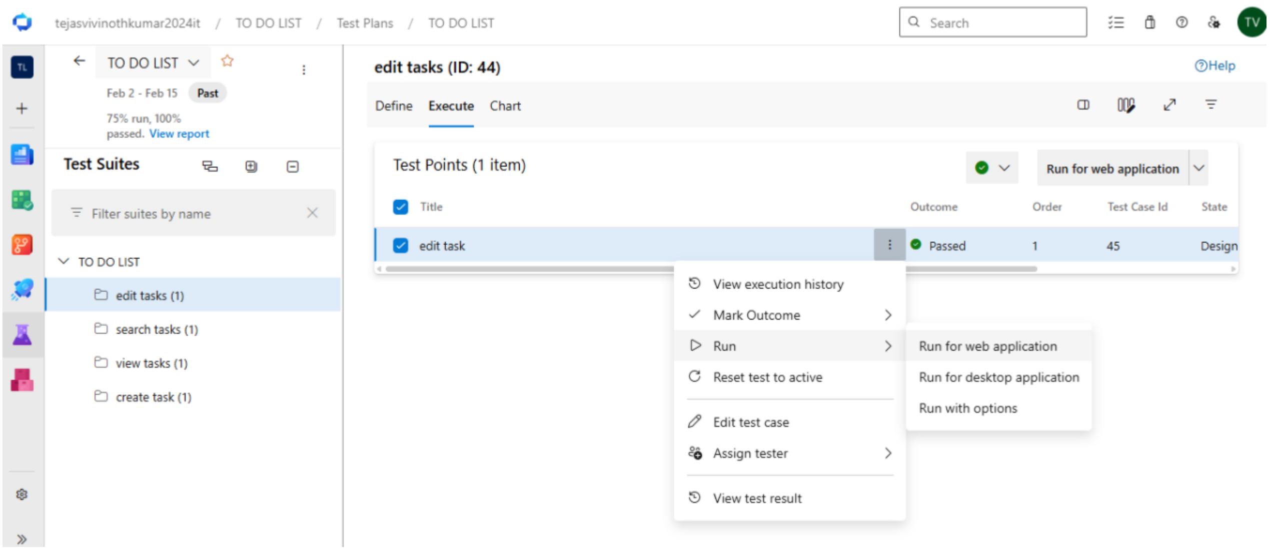
4.Installation of test

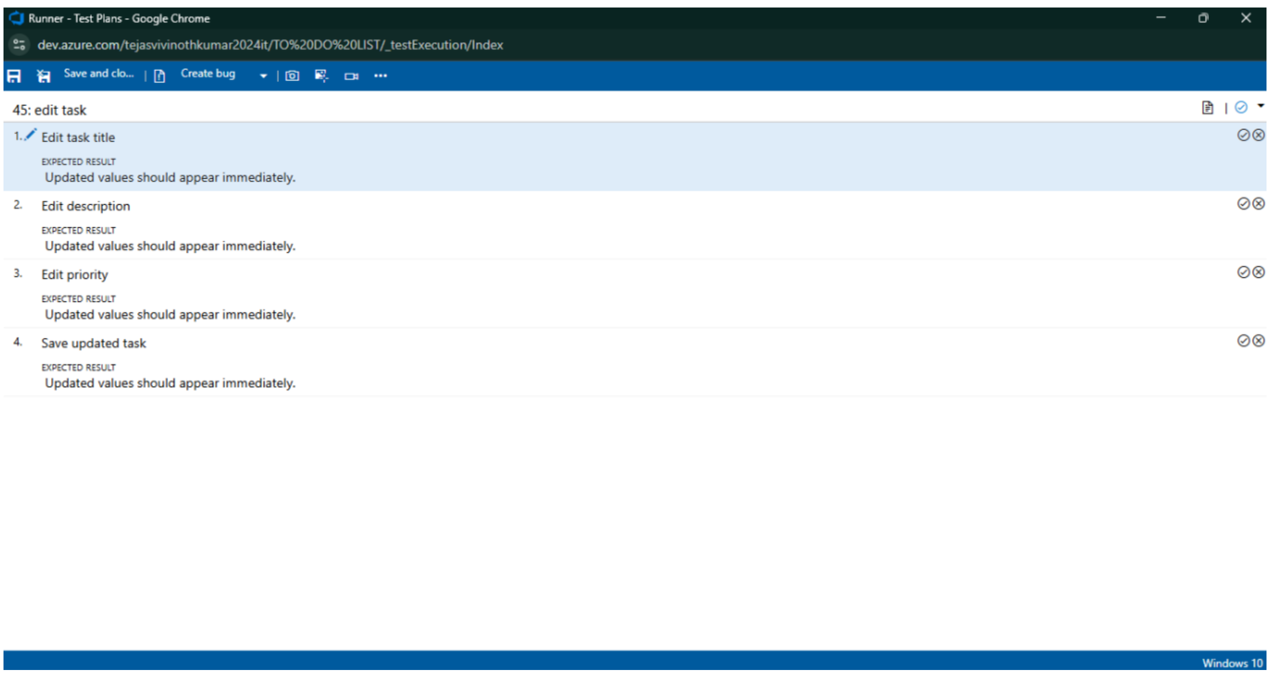


Test and feedback
Showing it as an extension

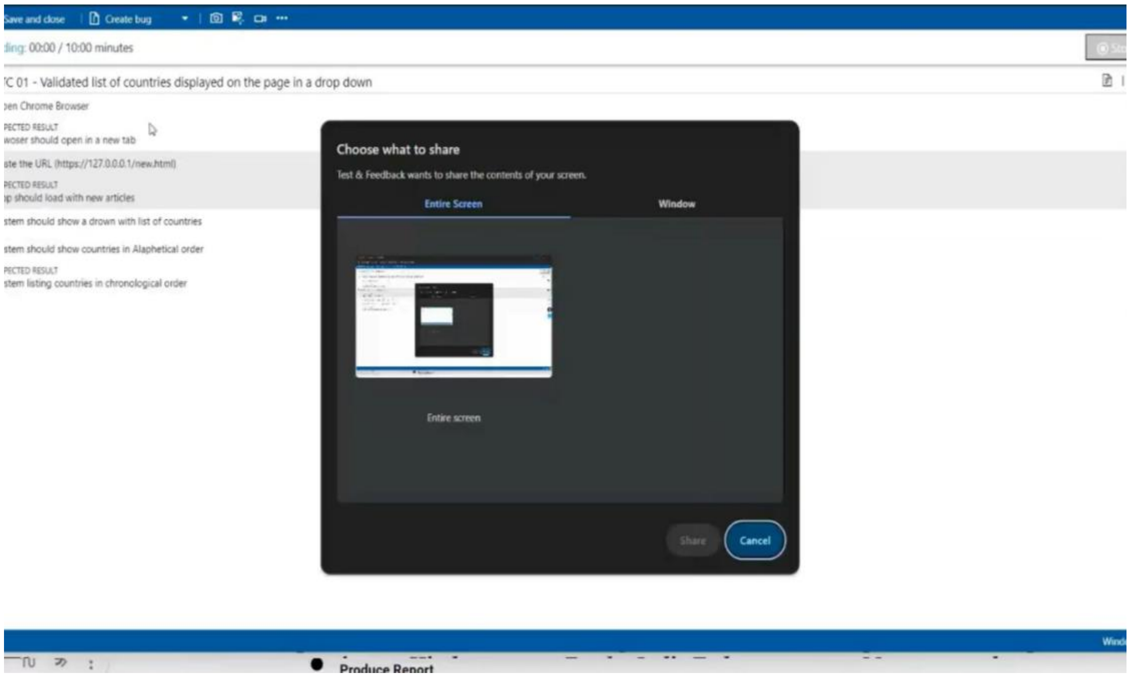


5.Running the test cases

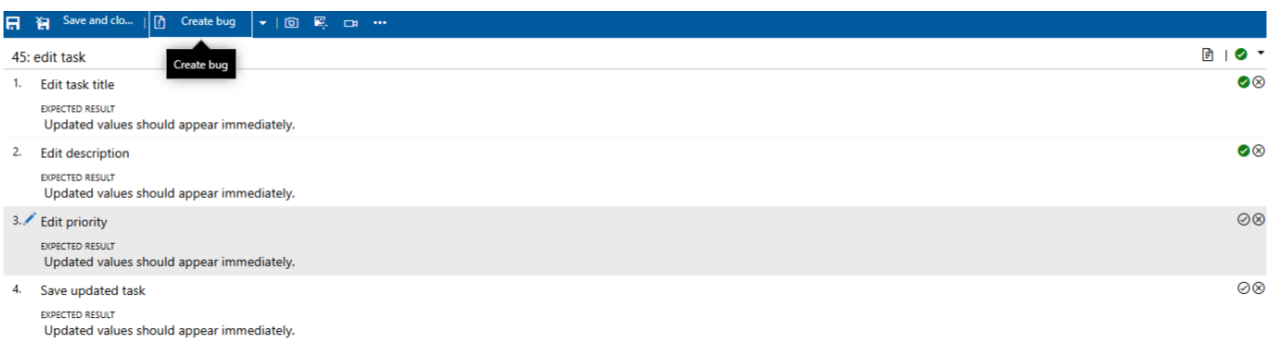


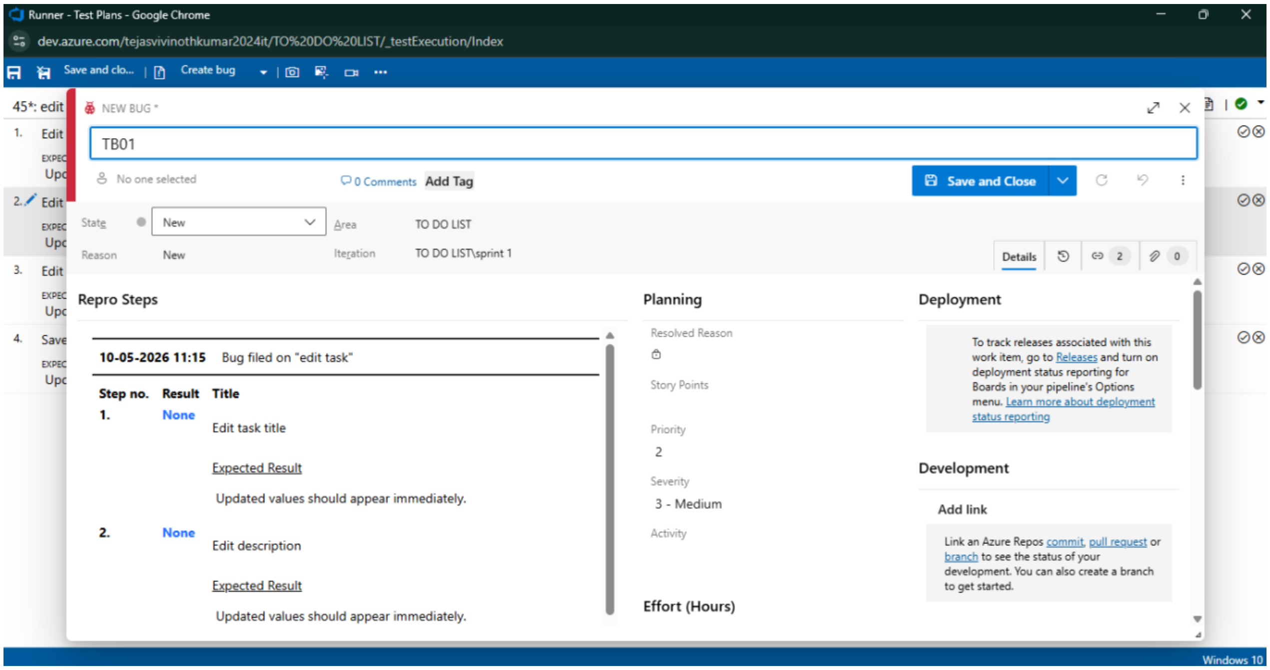


6.Recording the test case

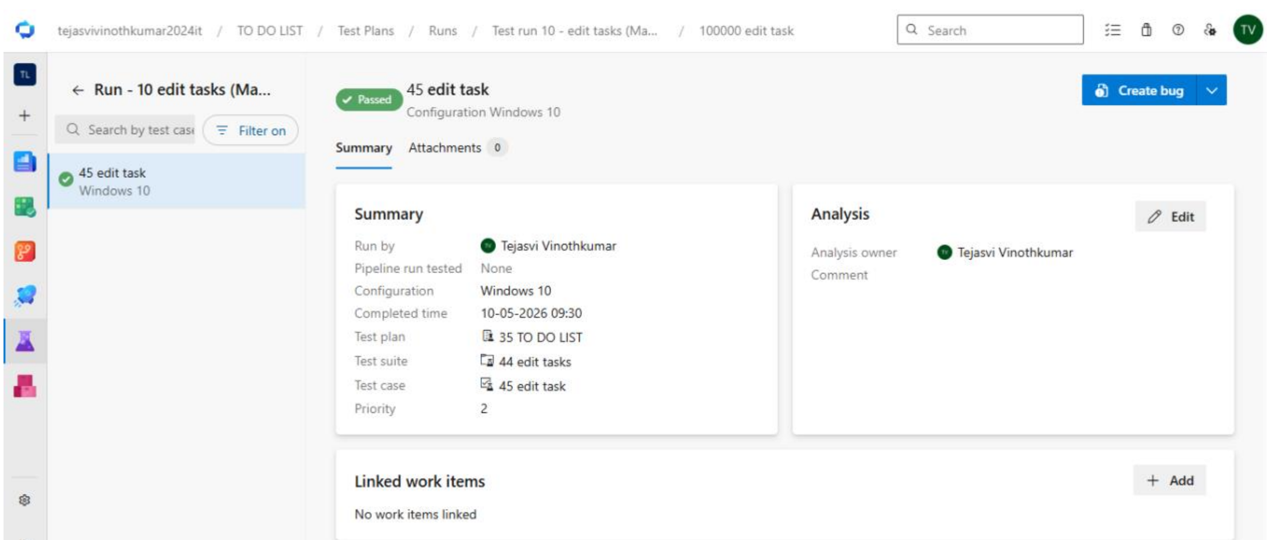


7.Creating the bug

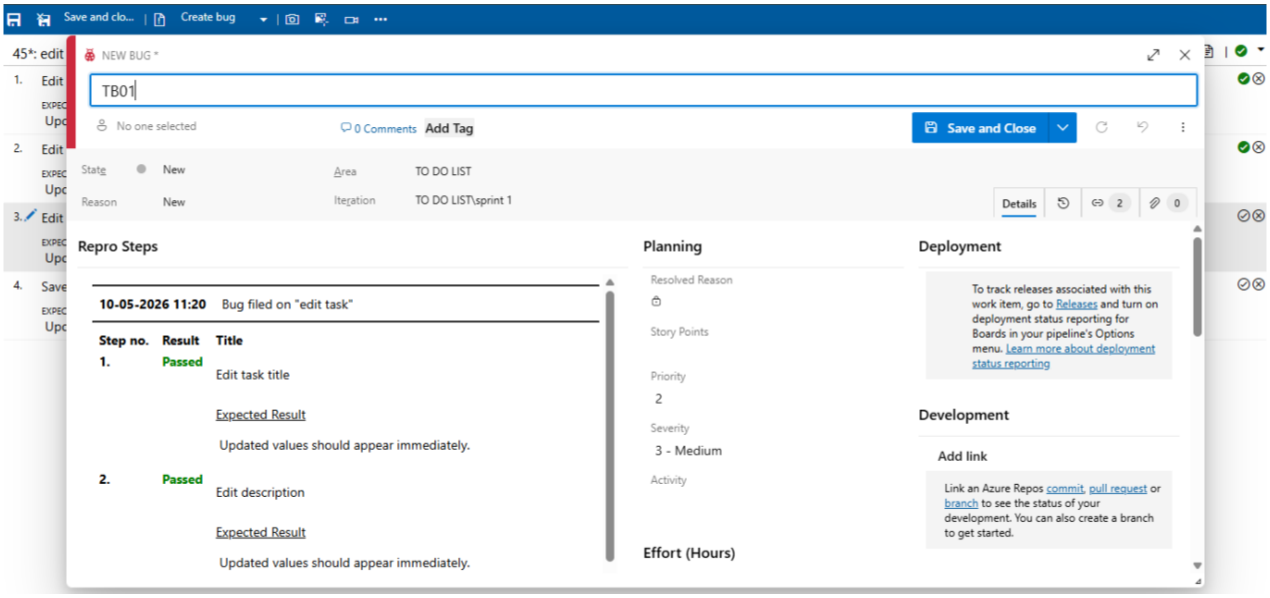




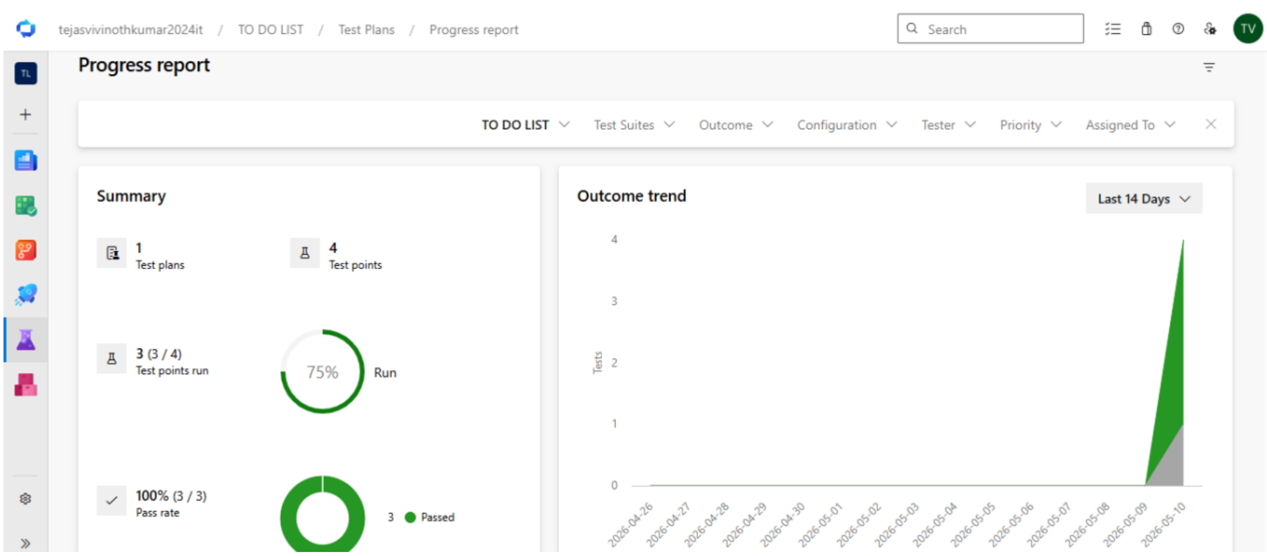
8.Test case results



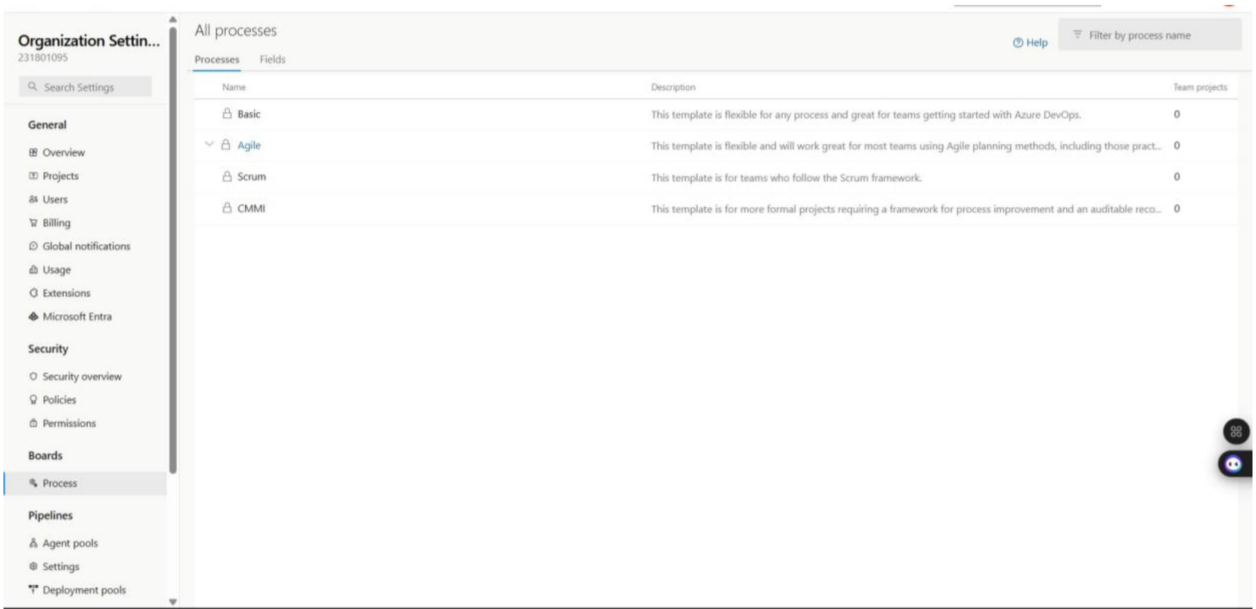
9.Test report summary



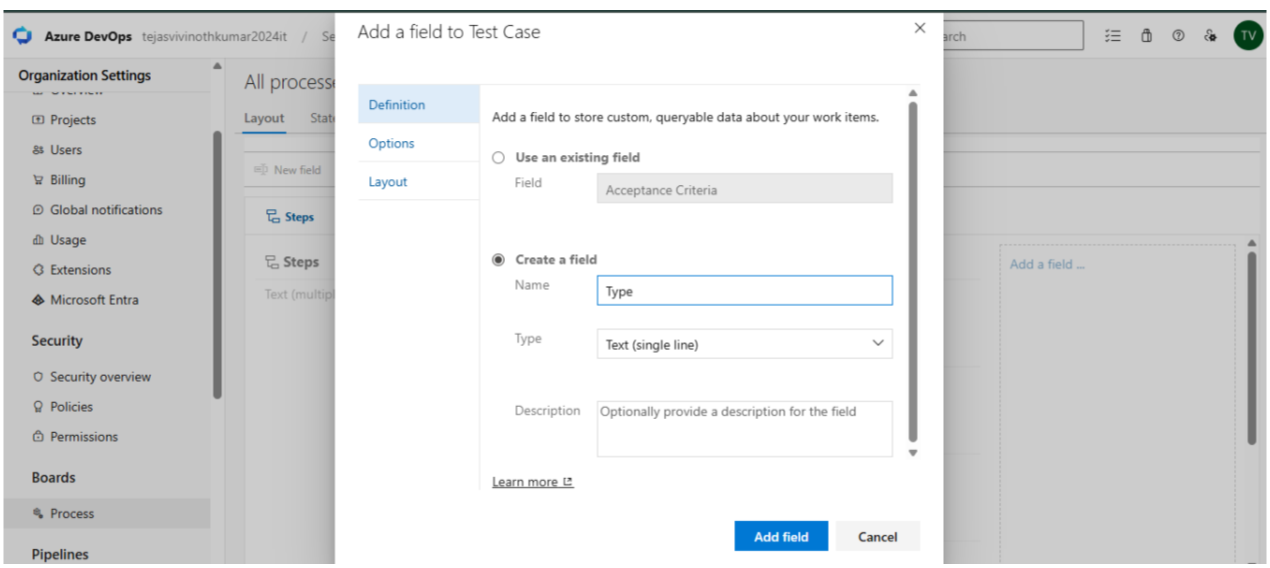
10.Progress report

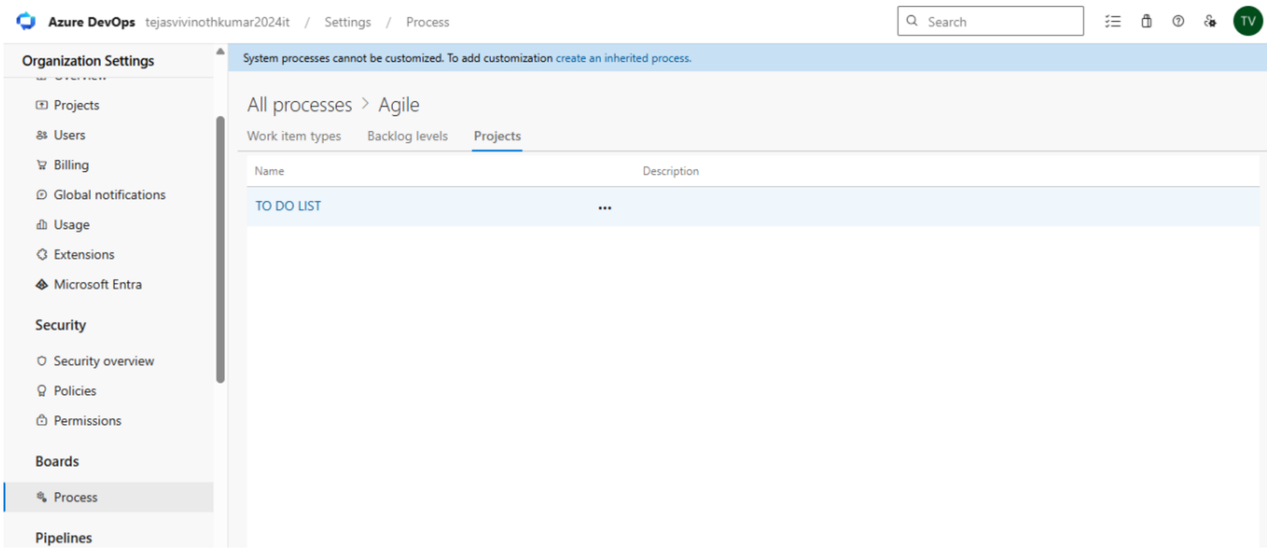


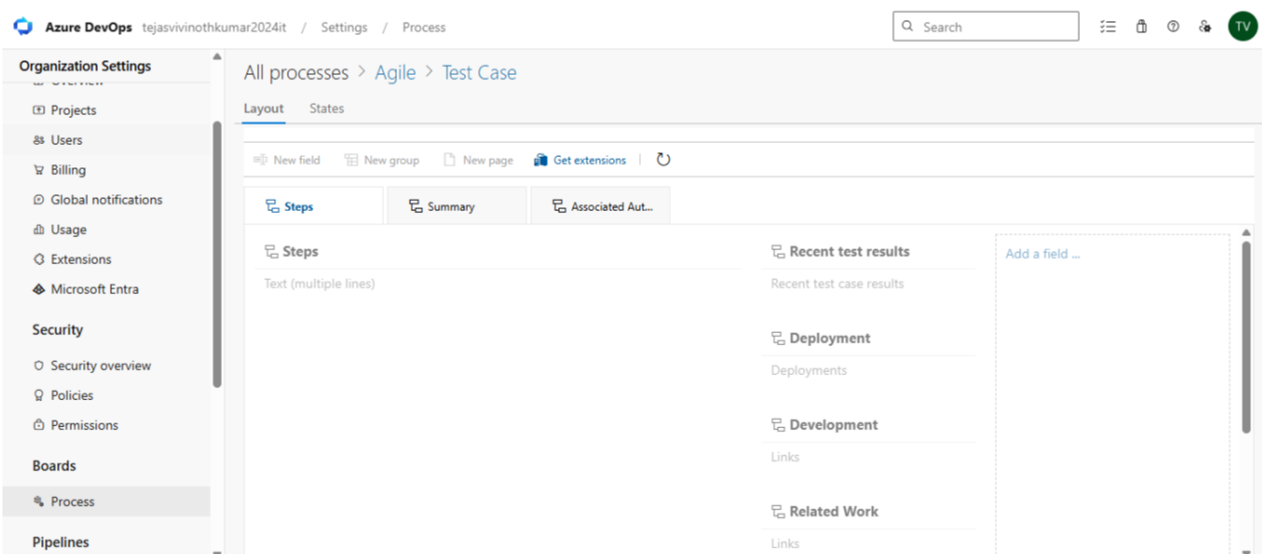
11.Changing the test template



12.View the new test case template







Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path

LOAD TESTING AND PIPELINES

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Azure Load Testing:
Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

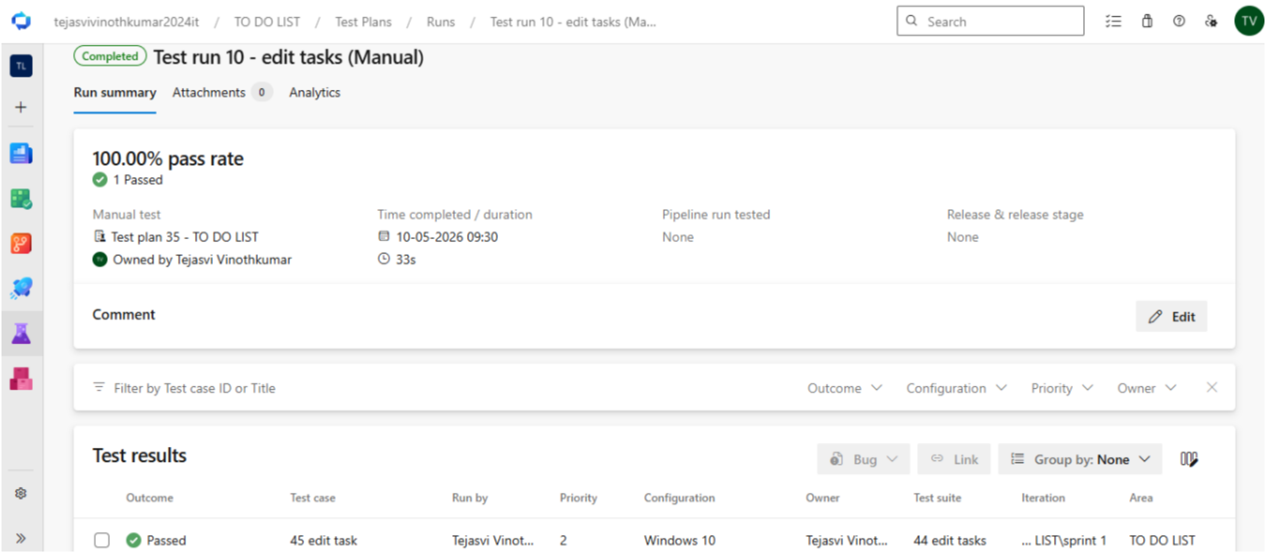
1. Sign in to Azure Portal
   Go to https://portal.azure.com and log in.
2. Create the Resource o Go to *Create a resource* → Search for "Azure Load Testing".
   - o Select Azure Load Testing and click Create.
3. Fill in the Configuration Details o Subscriptio*n:* Choose your Azure subscription. o *Resource Group:* Create new or select an existing one. o *Name:* Provide a unique name (no special characters).
   - o *Location:* Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

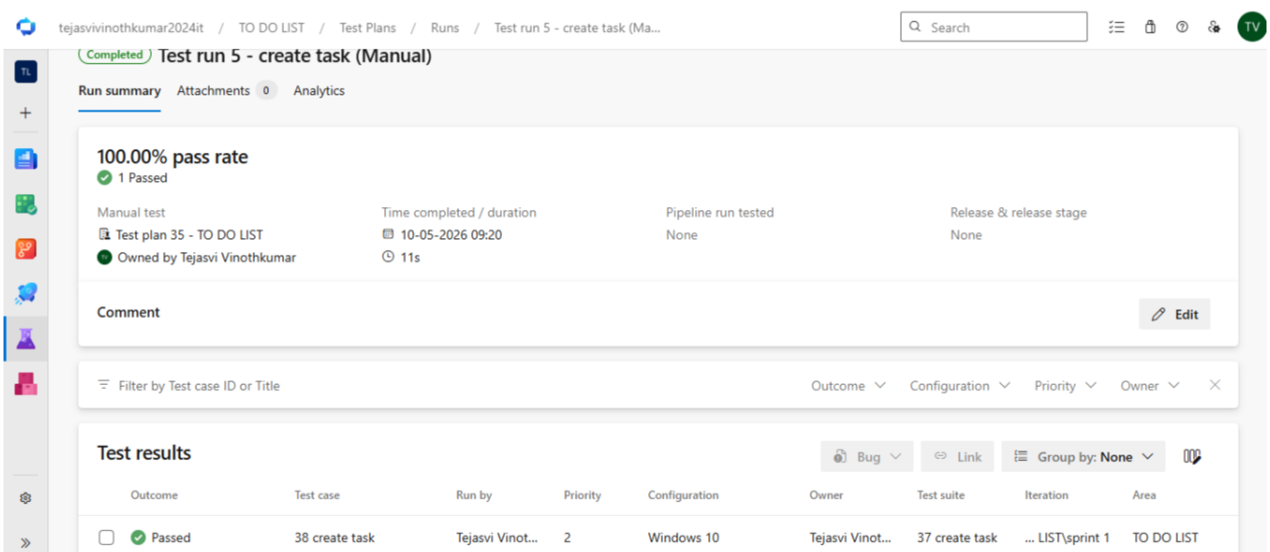
Steps to Create and Run a Load Test:

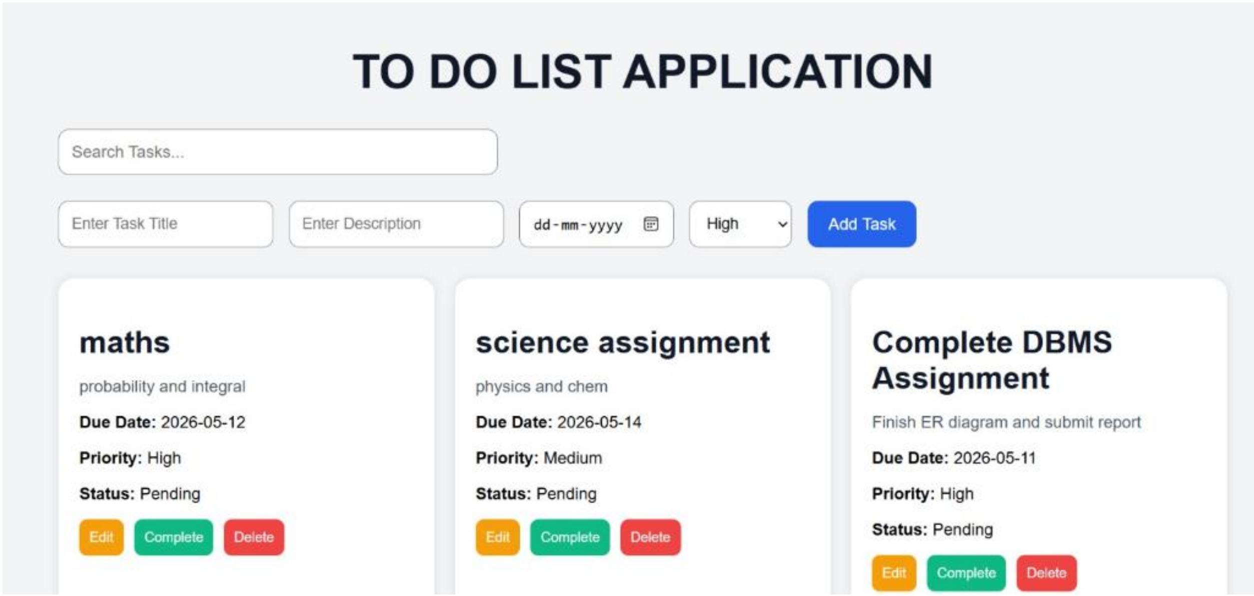
Once your resource is ready:

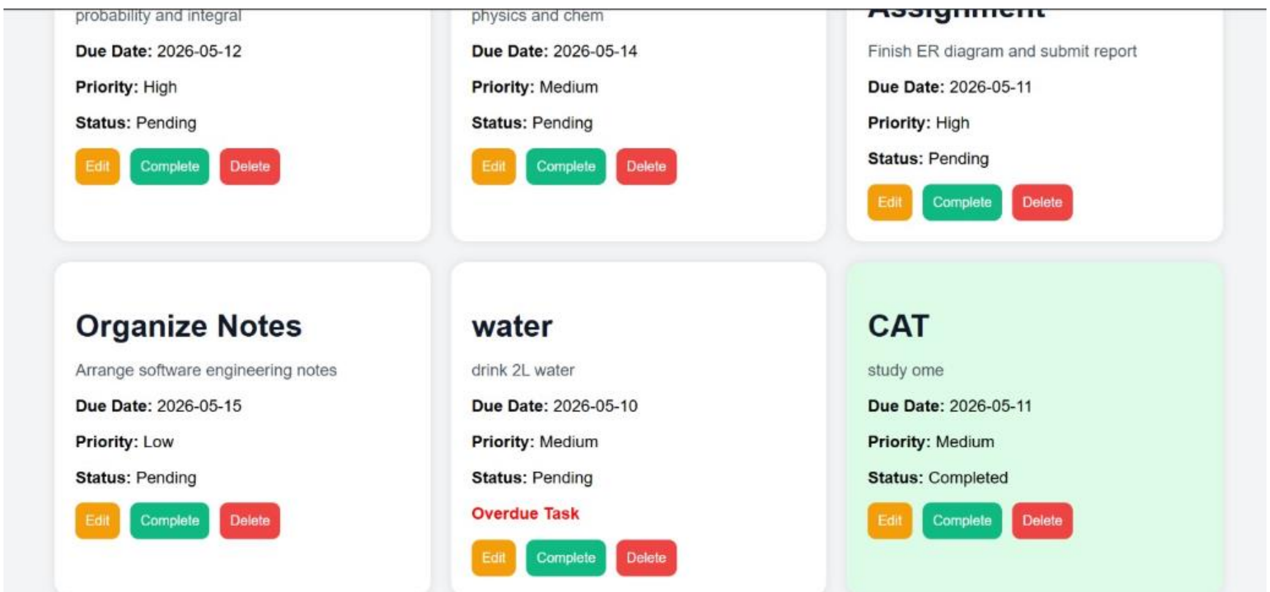
1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab o *Test Name:* Provide a unique name.
   - o *Description:* (Optional) Add test purpose.
   - o *Run After Creation:* Keep checked.
3. Load Settings o *Test URL:* Enter the target endpoint (e.g., https://yourapi.com/products).
4. Click Review + Create → Create to start the test.

Load Testing









Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Flask-based music recommendation project with Azure DevOps. The pipeline will automatically install dependencies, run basic tests, and publish artifacts. This ensures that every commit triggers checks for reliability and smooth deployment.

Steps:

1. Connect GitHub to Azure DevOps:
    - In Azure DevOps, create a new project.

- o Create a pipeline and select GitHub as the source.
- o Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.

2. Create azure-pipelines.yml in Your Repo Root:
    - o In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
    - o Add the following basic pipeline configuration for Python and Flask:

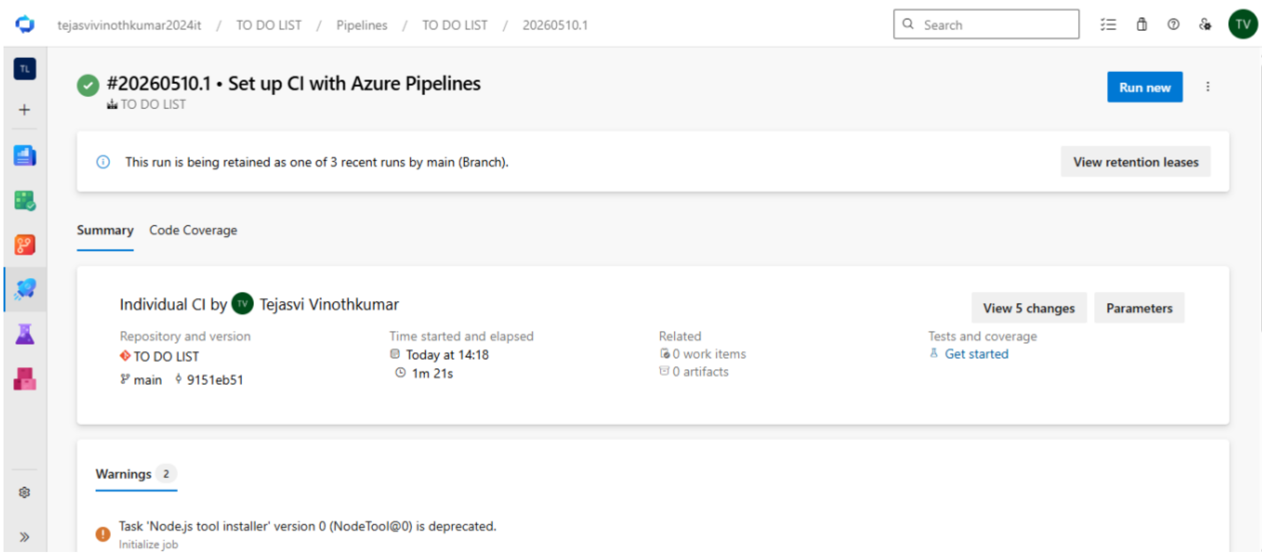
yml Code

```
trigger:
- main  # Trigger pipeline when changes are pushed to the main branch

pool:
  vmImage: ubuntu-latest  # Use a hosted Ubuntu agent

steps:
# Step 1: Checkout the code from GitHub
- checkout: self

# Step 2: Set up Python
environment - task:
UsePythonVersion@0   inputs:
    versionSpec: '3.x'  # Use the latest Python 3.x version
displayName: "Set up Python"

# Step 3: Install dependencies from the correct path
- script: |     python -m pip install --upgrade pip     pip install -r project/requirements.txt  #
  Adjusted path to requirements.txt   displayName: "Install dependencies"

# Step 4: Run a simple Python script to check the environment
- script: |     python -c "print('🎶 Hello from Music Playlist Batch Creator!')"
  displayName: "Run a Python script"
```

3. Pipeline Tasks Include:
    - o Setting up the Python environment using the UsePythonVersion task.
    - o Installing project dependencies from project/requirements.txt. Make sure the path to requirements.txt is correct (it is located under the project folder).
    - o Running a simple Python script to verify that Python is set up correctly and the pipeline works.
4. Run and Monitor Pipeline:
    - o Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.

- Monitor the logs in the Azure DevOps portal to view logs, errors, or success messages and ensure everything runs smoothly.

Pipeline



Result:

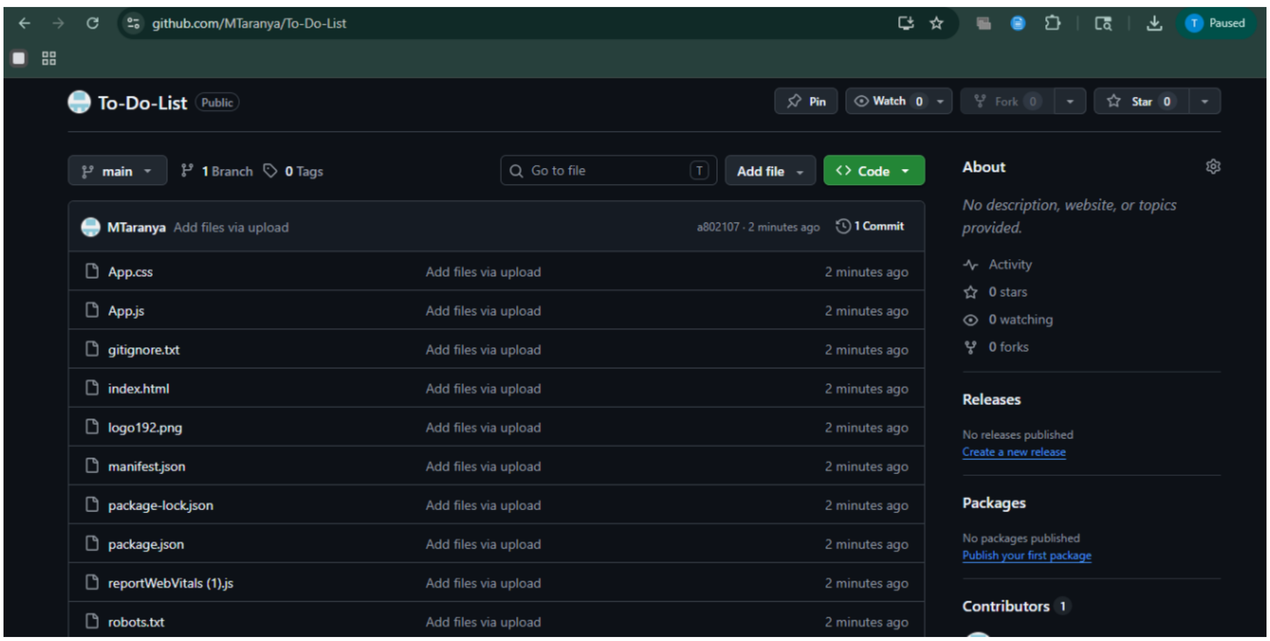
Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the To-Do List Application.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.